

DEVELOPING A TURKISH QUESTION ANSWERING SYSTEM WITH DEEP LEARNING MODELS

KADIR TUTAR

ÖZYEGİN UNIVERSITY

JANUARY, 2026

DEVELOPING A TURKISH QUESTION ANSWERING SYSTEM WITH DEEP LEARNING MODELS

by
Kadir Tutar

Dissertation
Submitted in Partial Fulfillment of the
Requirements for the Degree of

Doctor of Philosophy

in
Computer Science

Advisor: Prof. Olcay Taner Yıldız

Graduate School of Science and Engineering
Özyeğin University
İstanbul

January, 2026

DEVELOPING A TURKISH QUESTION ANSWERING SYSTEM WITH DEEP LEARNING MODELS

Approved by:

Prof. Olcay Taner Yıldız, Advisor
Department of Artificial Intelligence and
Data Engineering
Özyeğin University

Prof. Fikret Gürgen
Department of Computer Science
Boğaziçi University

Prof. Hasan Sözer
Department of Computer Science
Özyeğin University

Asst. Prof. İlknur Karadeniz
Department of Artificial Intelligence and
Data Engineering
Özyeğin University

Prof. Arzucan Özgür
Department of Computer Science
Boğaziçi University

Approval Date: December 26, 2025

To my spouse, my daughter, and my family for their love and patience, as well as my advisor, whose guidance shaped this dissertation.



DECLARATION OF ORIGINALITY

I hereby declare that I am the sole author of this thesis and that this is the true copy of my thesis, including the final revisions, approved by my thesis committee. All data and information have been obtained, produced, and presented in accordance with the rules of research ethics and principles of academic honesty. As required by these rules, to the best of my knowledge I have acknowledged ideas, thoughts, and any copyrighted material in accordance with the standard referencing rules. I certify that any part of this thesis has not been submitted for a degree or diploma in another educational institution.

Kadir Tutar

ABSTRACT

The field of Natural Language Processing presents several challenges, including question answering, which encompasses understanding reading comprehension, multiple-choice, yes-no, and other question types. A significant challenge in these studies is the creation of question answering datasets. While there are many examples of question answering datasets in English, they are limited, especially in languages like Turkish. To address this issue, the study developed a Turkish question answering dataset derived from the comprehensive English Stanford Question Answering Dataset (SQuAD) using a machine translation tool. Translator outputs were evaluated, and the data were then refined to increase the dataset size. Consequently, 78% of the SQuAD was effectively converted into Turkish.

Another challenge is the success of deep learning models. We developed baseline models using deep learning techniques on the newly created dataset and conducted analyses from multiple perspectives. The strengths and weaknesses of the models were examined through detailed error analysis. When our dataset was structured as a knowledge graph, our models were trained on it and run with it. We have shown that the structuralization and enrichment of the data contribute significantly to the model's success (EM and F1: +10% accuracy compared to other baselines). This approach enabled us to enhance the learning capacity of the system without altering the underlying closed-box architecture of the large language models and deep learning systems employed.

Keywords: Natural Language Processing, Question Answering, Machine Reading Comprehension, Turkish Question Answering Dataset, Artificial Intelligence

ÖZET

Doğal Dil İşleme çalışma alanı; okuduğunu anlama, çoktan-seçmeli, evet-hayır ve diğer soru tiplerini kapsayan soru-cevap alt alanı altında çözülmesi zor birçok problemi içermektedir. Bu alandaki önemli engellerden bir tanesi de Türkçe gibi dillerde veri setlerinin kısıtlı olmasıdır. Bu çalışmada, makina çevirisi kullanılarak İngilizce SQuAD veri setinden çevrilen bir Türkçe soru-cevap veri seti geliştirilmiştir. Tüm çeviri çıktıları değerlendirilerek manuel ve otomatik yöntemler ile SQuAD veri setinin %78'i soru cevap sistemlerin geliştirilmesi amacıyla Türkçe'ye çevrilmiştir.

Bir diğer zorluk ise yapay zeka derin öğrenme modellerinin soru-cevap sistemler üzerindeki performansdır. Yeni oluşturulan veri seti ile farklı derin öğrenme modelleri geliştirilmiştir. Modellerin sonuçları doğal dil işleme perspektifleri ile farklı yönlerden analiz edilerek modellerin güçlü ve zayıf yönleri birçok açıdan değerlendirilmiştir. Modellerin verinin derinliğine bağlı olarak başarılarının da değiştiği tespit edilmiştir. Veriler derin öğrenme ile çizge veri yapısı üzerinde yeniden yapılandırılarak modellerin yapısal veri seti ile eğitimi ve çalışması sağlanmıştır. Verilerin yapılandırılmasının ve zenginleştirilmesinin modelin başarısına önemli ölçüde katkıda bulunduğu tespit edilmiştir. Geliştirilen yeni modeller, önceki temel modellere kıyasla EM ve F1 üzerinde +%10 varan daha iyi performanslar elde edilmiştir. Bu yaklaşım sayesinde, büyük dil modellerinin ve derin öğrenme modellerinin kapalı-kutu mimarisi değiştirilmeden başarı oranı arttırılmıştır.

Anahtar Kelimeler: Doğal Dil İşleme, Soru Cevap Sistemler, Makine Okuma Anlama, Türkçe Soru Cevap Veri Seti, Yapay Zeka

ACKNOWLEDGEMENTS

I would like to express my profound gratitude to my advisor, Prof. Olcay Taner Yıldız, for his invaluable guidance, mentorship, and continuous encouragement throughout my doctoral journey. His insightful feedback and unwavering support have played a central role in shaping the direction and quality of this dissertation. I am also deeply thankful to the permanent jury members, Prof. Hasan Sözer and Prof. Arzucan Özgür, whose expertise, constructive comments, and thoughtful evaluations greatly contributed to the improvement of this work.

I would also like to acknowledge Özyeğin University for providing the financial support that made this research possible. The resources, academic environment, and opportunities offered by the university were essential in enabling the successful completion of this study. I am sincerely grateful for their commitment to fostering high-quality research and innovation.

To my daughter, Zeynep, who was born during this academic journey, you have been my greatest inspiration and motivation. Your love gave me the strength to continue and complete this work.

Finally, I would like to extend my heartfelt gratitude to my wife and family for their endless patience, love, and unwavering support. Their encouragement during challenging moments and their belief in my abilities have been a constant source of strength throughout this doctoral process. This accomplishment would not have been possible without them.

TABLE OF CONTENTS

DECLARATION OF ORIGINALITY	iv
ABSTRACT	v
ÖZET	vi
ACKNOWLEDGEMENTS	vii
LIST OF TABLES	xi
LIST OF FIGURES	xiv
LIST OF ACRONYMS AND ABBREVIATIONS	xv
1. INTRODUCTION	1
2. RELATED WORKS	6
2.1 Dataset Studies	6
2.2 QA Deep Learning Model Studies	9
3. INFRASTRUCTURE AND FEASIBILITY STUDIES	14
3.1 The Evolution of AI in NLP	14
3.2 Evaluation and Selection of Dataset	15
3.2.1 Evaluation Methodology and Datasets	15
3.2.2 Comparison between Datasets	18
3.3 Determining the Model Methodology & Fail Cases	20
3.3.1 Dependency Parser Similarity	20
3.3.2 NER Similarity	23
3.3.3 Semantic Similarity	24
3.3.4 Morphological Analysis & Data Summarization	26
3.4 End-to-end Pre-Studies & Trials	27
3.4.1 Creation Pre-Study Dataset	27
3.4.2 Assessment of Deep Learning Approaches	30
3.4.3 Evaluation Metrics	34

3.4.4	Pre-evaluation Deep Learning Approaches	35
4.	CREATING DATASET	37
4.1	SQuAD Dataset	37
4.2	The Creation of mSQuAD	39
4.3	Post Translation Actions	40
4.4	Statistics of mSQuAD	44
4.5	Evaluation Dataset with Deep Learning Model.....	45
4.5.1	BERT.....	45
4.5.2	Fine-Tuned Model	47
4.5.3	Results and Evaluation	47
5.	CREATING BASELINE MODELS AND ANALYSIS OF RESULTS	54
5.1	Bidirectional Encoder Representations Transformer Baseline.....	54
5.2	OpenAI ChatGPT & Qwen2 Baselines	54
5.3	Baseline Results	58
6.	EXPERIMENTS WITH STRUCTURED AND ENRICHED DATA.....	63
6.1	Creating Structured Paragraph Data.....	63
6.2	Creating Enriched Question Data	69
6.3	Training with Structure Data	73
7.	RESULTS OBTAINED WITH STRUCTURED AND ENRICHED DATASETS .	76
7.1	Results by Content Length.....	79
7.2	Results by Domain	81
7.3	Results by Question Type.....	82
7.4	Error Analysis	84
7.5	Performance and Cost Analysis	92
7.6	Alternative Dataset Performance	93
8.	COLLABORATIVE WORK WITH LINGUISTIC RESEARCHERS	95
8.0.1	Building the Corpus of the Treebank	95
8.0.2	Annotation	99

9. CONCLUSIONS	102
REFERENCES	104
VITA	108



LIST OF TABLES

Table 3.1:	Benchmark comparison of MRC datasets	18
Table 3.2:	Dependency parser similarity results	22
Table 3.3:	NER-based similarity results	24
Table 3.4:	Semantic similarity results	25
Table 3.5:	Facebook bAbI project question and answer task list	29
Table 3.6:	Facebook bAbI project QA task data format	29
Table 3.7:	Comparison of RNN, LSTM, GRU, and BERT models	33
Table 3.8:	Exact Match (EM) evaluation examples	34
Table 3.9:	F1-score evaluation examples	35
Table 3.10:	EM Performance of RNN, LSTM, GRU, BERT for the first three bAbI tasks	36
Table 4.1:	The statistics of SQuAD	38
Table 4.2:	The sample data of SQuAD	38
Table 4.3:	The sample data of mSQuAD	41
Table 4.4:	Effect of the post-translation correction type to translated answers	44
Table 4.5:	The translation output statistic of mSQuAD.....	45
Table 4.6:	Needed human operations after translation	46
Table 4.7:	SQuAD, mSQuAD and gSQuAD Dataset Question/Title/Paragraph Count Statistics.	49
Table 4.8:	m-QA-Model-Tr and g-QA-Model-Tr models results	50
Table 4.9:	m-QA-Model-Tr results on mSQuAD-dev-tr with tokenize count perspective as c =content, q= question, a= answer	50
Table 4.10:	m-QA-Model-Tr results on mSQuAD-dev-tr with domain oriented, top-5, bottom-5	51

Table 4.11: g-QA-Model-Tr results on mSQuAD-dev-tr with domain oriented, top-5, bottom-5	52
Table 4.12: m-QA-Model-Tr results on mSQuAD-dev-tr with question type, q type count ≥ 100	52
Table 4.13: g-QA-Model-Tr results on mSQuAD-dev-tr with question type, q type count ≥ 100	53
Table 4.14: m-QA-Model-Tr results with BLEU as c =content, q= question, a= answer	53
Table 5.1: Baselines prompt samples	58
Table 5.2: Results of baseline models on mSQuAD	59
Table 5.3: BERT, ChatGPT, Qwen2 baselines with token-count (c =content, q= question, a= answer), Baseline = B, Token Size = #, Token Type = T	60
Table 5.4: Domain based evaluation - BERT, ChatGPT, Qwen2 Baselines, Base-line= B	61
Table 5.5: BERT, ChatGPT, Qwen2 baselines with question type, samples from question type count ≥ 100 , Baseline= B	61
Table 6.1: Evaluation of knowledge base creation technique, ranking(1 to 10, Low:0-3, Medium:4-7, High:8-10).....	65
Table 6.2: Transformation of structured data sample	68
Table 6.3: Results of question enrichment trial methods	71
Table 6.4: Accuracy of different prompts on Baseline 1 for A alternative question..	71
Table 6.5: Baseline ChatGPT prompt structure	72
Table 6.6: Sample of enriched question data	72
Table 7.1: Performance of BERT Baseline, ChatGPT Baselines, QModel (Question Enrichment), KBModel (Structured Content + Question Enrichment)	77
Table 7.2: KB model vs baselines with tokenize count perspective as c =content, q= question, a= answer	80
Table 7.3: KB model vs baselines - Domain based evaluation	84
Table 7.4: KB model vs baselines with question type perspective, q type count ≥ 100	84

Table 7.5: Performance of models 8K question call for test dataset	93
Table 7.6: Performance of KBModel on TQuADv1 dataset and comparison of its baselines	94
Table 8.1: Distribution of questions in the corpus.....	98
Table 8.2: Phrase groups and examples for the most 10 frequent question types	100



LIST OF FIGURES

Figure 1.1: Example of structural differences between Turkish and English.	1
Figure 3.1: Facebook bAbI examples	28
Figure 4.1: Trained two QA models (m-QA-Model-Tr, g-QA-Model-Tr) based on mSQuAD and gSQuAD	48
Figure 5.1: BERT Baseline, fine-tune operation with BERTurk on mSQuAD Dataset 55	
Figure 6.1: Process of structuring data transformation	66
Figure 6.2: Sample of knowledge graph data	67
Figure 6.3: BERT Training with Structured Data	74
Figure 7.1: Performance of BERT baseline, ChatGPT baselines, Qwen2 baseline, QModel (Question Enrichment), KBModel (Structured Content + Question Enrichment)	78
Figure 8.1: The distribution of word counts of questions in Squad and HaydiSor ...	96
Figure 8.2: All processes are shown end to end	97
Figure 8.3: The interface used to mark the dependency relations	101

LIST OF ACRONYMS AND ABBREVIATIONS

AI Artificial Intelligence

NLP Natural Language Processing

QA Question Answering

SQUAD Stanford Question Answering Dataset

MRC Machine Reading Comprehension

BERT Bidirectional Encoder Representations from Transformers

EM Exact Match

RNN Recurrent Neural Network

LSTM Long Short-Term Memory

GRU Gated Recurrent Unit

GPT Generative Pre-trained Transformer

NER Named Entity Recognition

SPARQL Protocol and RDF Query Language

ARC AI2 Reasoning Challenge

GAI Generative Artificial Intelligence

IR Information Retrieval

IEEE Institute of Electrical and Electronics Engineers

LLM Large Language Model

BLEU Bilingual Evaluation Understudy

UBMK International Conference on Computer Science and Engineering

KB Knowledge Base

AGI Artificial General Intelligence

SBERT Sentence BERT

CoNLL Conference on Computational Natural Language Learning

LREC Language Resources and Evaluation Conference

SIU Sinyal İşleme ve İletişim Uygulamaları

OPUS Open Parallel Corpus

RACE Large-scale Reading Comprehension Dataset From Examination

SQuAD Stanford Question Answering Dataset

CLS Classify Token

SEP Seperator

TR Turkish

EN English

NP Noun Phrase

ADJP Adjective Phrase

PP Prepositional Phrase

ID Identity

QNO Question No

VBZ Croatian Publishing Company

ACM Association for Computing Machinery

NFC National Football Conference

NFL National Football League

WBT Jefferson Standard Broadcasting Company



1. INTRODUCTION

Turkish is an agglutinative language in which words are typically formed by starting with a base root and sequentially attaching multiple suffixes. These suffixes can encode information about tense, aspect, mood, person, number, possession, and case. All of them might require several separate words or auxiliary constructions in English. As illustrated in Figure 1.1, an expression that would need a whole phrase or clause in English, in many cases, is represented by just one complex word in Turkish. These substantial structural differences between the two languages mean that direct, unmodified transfer of methods, models, or linguistic resources from English to Turkish can be unreliable or suboptimal. Tokenization and word-level approaches often need to be redesigned to account for Turkish morphological features. Because of these cross-linguistic variations, assumptions of applied Natural Language Processing (NLP) methods may not carry over from one language to another. For this reason, it is crucial to evaluate and refine solutions for each language separately.

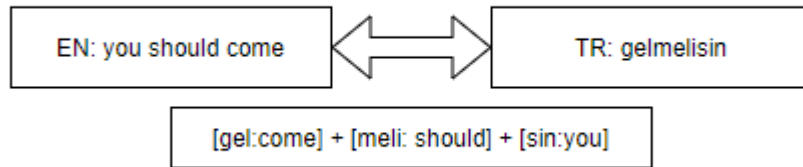


Figure 1.1: *Example of structural differences between Turkish and English.*

Question answering (QA) is a challenging field in NLP. It requires systems to understand, interpret, and reason over human language. Within this broad field, one of the most active and essential sub-areas is Machine Reading Comprehension (MRC). MRC tasks are specifically designed to evaluate a model's ability to read a given text and identify the answer to a question directly from the content of that text, rather than relying

on external knowledge sources. In other words, the answer must be found within the boundaries of the provided document or passage.

One significant barrier in academic research on MRC is the lack of datasets in low-resource languages. This problem is especially pronounced in Turkish. Existing resources are sparse, fragmented, and often unsuitable for training or evaluating modern deep learning models. Creating a dataset for QA systems is not a simple task, and it is an effort-intensive process that involves careful design, data collection, annotation, quality control, and validation. This type of work usually requires significant time and coordination across research teams and domain experts.

Several well-established datasets exist for English. One of them is SQuAD, introduced by [1], which is a large-scale, open-domain MRC dataset in English. SQuAD has played a foundational role in advancing QA research. Some deep learning-based architectures, transformer models, and pre-trained language models were applied and tested on SQuAD. As a result, SQuAD has effectively served as a standard testbed for comparing methods in the development of robust QA systems.

To address the lack of Turkish-language datasets, our preliminary study aimed to build one. We focused on translating the SQuAD dataset into Turkish using state-of-the-art machine translation techniques. The process did not end with automatic translation. After the initial machine-generated output, we conducted extensive manual post-editing and revision. These human interventions were crucial for correcting translation errors, resolving ambiguities, and ensuring that questions remained answerable from the corresponding passages.

After finalizing the translated and revised dataset, we performed a series of experiments to assess its utility and quality. In particular, we evaluated the consistency, reliability, and overall performance of the new Turkish MRC dataset using baseline deep

learning models that are commonly employed in QA research. These experiments allowed us to analyze how well such models learn from the dataset and provided an initial benchmark for future studies on Turkish QA and MRC.

More comprehensive descriptions of the dataset creation process, the translation pipeline, the manual editing criteria, and the experimental findings, along with detailed quantitative results, are provided in our publication. Our study is [2], which provides additional analyses, discusses limitations, and offers suggestions for future work on Turkish MRC. 75% of the original dataset has been converted into Turkish for use in training question-answer systems.

In recent years, transformer-based architectures have enabled the use of attention mechanisms in NLP. These architectures are specifically designed to capture long-range contextual dependencies. As a result, they produce rich, high-dimensional language representations that encode subtle semantic and syntactic information. Researchers, developers, and companies have developed Large Language Models (LLMs) to understand context using transformer architectures.

Our study reveals that the performance of our baselines is far from uniform. Its accuracy varies substantially across domains. We found that while the model achieves very high accuracy in specific domains, it often does so in those featuring more straightforward language and less specialized content. Its performance is decreasing in scientifically oriented areas. High conceptual density, complex sentence structures, and extensive use of technical jargon typically characterize these science-related domains. All of the observations appear to pose additional challenges for the model.

Furthermore, our analysis suggests that it is not only the domain itself but also the inherent complexity and formulation of the questions that influence the model’s ability to identify the correct answers. Questions that require multiple reasoning steps, that

involve domain-specific terminology, or that demand precise interpretation of subtle distinctions tend to be more difficult for the model to handle correctly. We also observe that the model’s accuracy tends to decline as the length of the question and its associated context paragraph increases. Longer texts generally introduce more information, potential distractions, and intricate dependencies, which seem to make it harder for the model to consistently focus on the most relevant evidence and arrive at the correct answer. These observations highlight the importance of domain characteristics, question complexity, and input length when evaluating models. Even though a single model can often perform reasonably well across a broad range, its performance may deteriorate noticeably when any of these conditions change.

In addition to how data influences models in this way, many developers are attempting to reconstruct an LLM from scratch or design transformer-based architectures. Such tasks often involve complex model configuration, hyperparameter tuning, large-scale data preprocessing, and advanced optimization strategies. All of them demand time, expertise, and infrastructure. As a result, directly modifying or rebuilding these models is often impractical or unsustainable for most teams and organizations.

We studied transforming the data to address these challenges for developing a more balanced, robust. We focused on the structural transformation of questions and paragraphs, reorganizing and reformulating the input data to better align with the model’s strengths and inductive biases. This data-centric approach aims to improve the model’s generalization. After applying structural transformations to the training data, we observed notable improvements in the models’ learning capabilities. The models captured relationships within the data more effectively, adapted to more variations, and across different domains and question types. We focused on analyzing these empirical findings to examine how the transformed data affected learning dynamics. This analysis confirmed that carefully structured modifications to the data can significantly improve model robustness

and adaptability. In our [3] study, we elaborate on the methodological approaches, setup, evaluation metrics, and comparative analyses.

The thesis is organized as follows. *Chapter 1 - Introduction* introduces the research problem, motivation, and main contributions of the study. *Chapter 2 - Related Works* reviews related works in the literature, highlighting existing approaches and identifying the research gap addressed in this work. *Chapter 3 - Infrastructure and Feasibility Studies* presents the infrastructure and feasibility studies, detailing the technical environment, tools, and preliminary analyses that guided the research design. *Chapter 4 - Creating Dataset* describes the process of creating the dataset, including data population, preprocessing, and annotation strategies. In *Chapter 5 - Creating Baseline Models and Analysis of Results*, baseline models are constructed and evaluated, and a detailed analysis of their performance is provided. *Chapter - 6 Experiments with Structured and Enriched Data* reports the experiments conducted using structured and enriched data, explaining the applied enrichment techniques and experimental setup. *Chapter-7 Results Obtained with Structured and Enriched Dataset* discusses the results obtained from these datasets, compares them with baseline performance, and analyzes their impact. Finally, *Chapter - 8 Collaborative Work with Linguistic Researchers* outlines the collaborative work carried out with linguistic researchers, emphasizing interdisciplinary contributions and their role in improving data quality and experimental outcomes.

2. RELATED WORKS

2.1 Dataset Studies

There are several categories of question–answering tasks in NLP, and one of the most prominent among them is MRC. In MRC tasks, the goal is typically to derive or extract precise answers from a given passage of text, such as a paragraph or a set of paragraphs, based on a user’s query. One of the most influential benchmarks in this domain is SQuAD, introduced by [1], which has become a standard open-domain MRC dataset in English.

Building an equivalent resource in another language requires substantial effort in translation quality control, annotation consistency, and dataset validation. SQuAD v1.1 contains more than 100,000 crowd-sourced questions. All resources derived from Wikipedia articles. Each question belongs to a paragraph. The answer is a span of text taken directly from that paragraph. The training split of SQuAD v1.1 includes 442 distinct article titles, while the test split consists of 48 titles. Every title has multiple paragraphs. In many cases, a single question can have multiple valid answer spans.

In one of our studies [2] ¹, we addressed the challenge of creating a similar resource for Turkish. We began translating the SQuAD dataset from English to Turkish using machine translation. This initial automatic translation step produced a comprehensive, noisy draft of a Turkish MRC dataset. We conducted post-editing and applied batch-processing operations, which include systematic error correction and formatting adjustments. As a result of these refinements, we reconstructed approximately 78% of the original English SQuAD dataset in Turkish.

Gemirter [4] previously produced a Turkish translation of the SQuAD dataset,

¹Our dataset GitHub link: <https://github.com/kadirtutar/datasetTranslated>

making approximately 50% of the original English data available in Turkish. In our earlier study [2], we systematically evaluated the effectiveness and reliability of our own translation methodology by using Gemirter’s dataset from [4] as a control dataset; in other words, it functioned as a benchmark against which we could compare the quality and characteristics of our translations.

In a separate line of research, Budur et al. [5] adopted a different approach, using Amazon Translator, a commercial machine translation service, to translate SQuAD2.0 into Turkish. In their work, they did not limit themselves to a subset of the corpus; instead, they translated all the crucial components of the original dataset, including article titles, context paragraphs, questions, and precise answer spans. As a result of this comprehensive translation effort, they obtained 25,528 question–answer pairs, in addition to 259 paragraphs that did not contain any associated question–answer pairs, as well as a further 3,582 question–answer pairs derived from the development portion of SQuAD2.0. Altogether, these translated elements correspond to roughly 50% of the original dataset being usable in Turkish. Their work made a substantial contribution to Turkish question–answering research by integrating this translated dataset. They not only provided a valuable resource for training and evaluation but also demonstrated how such a translated corpus can be effectively utilized in a realistic setting.

The THQuAD [6] dataset is a relatively recent resource explicitly developed for Turkish. It is designed to support research in open-domain QA. It adopts the same structural format and general design principles as the SQuAD dataset. Despite this similarity in format, THQuAD currently does not reach the same level of content diversity, scale, or thematic as SQuAD. It still represents an important and pioneering step toward building large-scale, Turkish-language resources for QA tasks. Its current version includes 15,554 questions, which is a non-trivial size for a specialized Turkish corpus. Also, the scope of THQuAD is constrained to just two main domains: Ottoman History and the History

of Turkish Islamic Science. This narrow domain coverage limits the dataset’s ability to represent the wide variety of topics fully.

Deep learning models require datasets that span a broad range of subjects, such as geography, contemporary events, the arts, and cultures. Because models may show noticeable reductions in performance when faced with specialized or less commonly represented domains, such as advanced philosophy, the history of science, religious studies, or niche scientific discipline. In this context, THQuAD highlights both the progress made in Turkish-language QA resources and the ongoing need to expand domain coverage and content variety to better support the training and evaluation of models capable of handling both general and highly specialized topics.

Similar investigations have been carried out for a range of other languages, aiming to adapt the original SQuAD dataset beyond English. For instance, [7] presented a detailed account of the development of a Spanish version of SQuAD. In their work, the authors relied on a dedicated, custom-built infrastructure to manage the entire translation pipeline, from the initial automatic translation of the English data to the subsequent processing steps. After the raw translations were produced, they implemented a series of tailored heuristics to refine the data: these heuristics were designed to filter out low-quality instances, correct evident translation errors, and better align questions, contexts, and answers in the target language. As a result of this multi-stage process, two distinct Spanish datasets were created. The first, SQuAD-es v1.1, is a larger resource that closely mirrors the scale of the original English SQuAD v1.1 but inevitably includes a higher level of noise due to its broader coverage. The second, SQuAD-es-small, contains only 53% of the original examples, making it considerably smaller, but, according to the authors’ analysis, notably more reliable and cleaner for rigorous experimentation.

A similar line of research has been pursued for French. In [8], the authors focused on producing a French adaptation of SQuAD by translating only a subset of the English

dataset. Their methodology involved not just translating isolated QA pairs but also carefully selecting and translating the corresponding paragraphs from which these pairs were derived. This process ensured that each French question and answer remained contextually grounded in a coherent French passage. Their efforts resulted in a collection of 327 (question, answer, paragraph) triplets drawn from 48 distinct articles. This resource provides valuable material for studying cross-lingual transfer and evaluating reading comprehension systems in French. Compared to the scale of the original SQuAD, the available evidence and empirical findings for French remain somewhat limited.

2.2 QA Deep Learning Model Studies

Neural network architectures have driven substantial improvements in the performance of QA systems. They enable rich linguistic representations and sophisticated attention mechanisms that focus on the most relevant parts of the input. The study of Vaswani et al. [9] introduced the self-attention mechanism, which explicitly models the idea that specific tokens or segments of an input sequence should be given greater weight than others. It provides a computational framework that enables the model to dynamically adjust these priorities during training and inference. This innovation laid the foundation for a new generation of transformer-based models that have reshaped the landscape of NLP.

Google released Bidirectional Encoder Representations from Transformers (BERT) in 2018 [10], which represents a significant milestone in the evolution of language models. BERT's bidirectional training on large text corpora enables it to capture contextual information from both the left and right of each token. These embeddings significantly improve downstream performance on QA benchmarks. Among these benchmarks, SQuAD has been widely adopted as a standard for evaluating MRC systems.

As reported in the current state-of-the-art results for SQuAD², BERT achieves an exact match (EM) score of 0.87 and an F1 score of 0.93 on the English version of the dataset. These metrics indicate that BERT not only frequently identifies the correct span of text as the answer (as reflected by the EM score) but also closely overlaps with the ground-truth answers at the token level (as captured by the F1 score). Consequently, BERT is regarded as one of the most influential architectures in advancing the state of the art for QA and has served as a foundation for many subsequent models and extensions.

Neural network approaches that leverage pre-trained language representations, among which ELMo [11] and the Generative Pre-trained Transformer (GPT) [12] are particularly notable. ELMo employs a deep, bidirectional architecture that processes text in both left-to-right and right-to-left directions. It enables the capture of rich contextual information from surrounding words. In contrast, GPT was designed as a unidirectional, transformer-based model that reads text from left to right. Its architecture introduced fewer structural changes when applied to different NLP tasks. This structure made it easier to integrate and fine-tune across a broad range of applications, such as text classification, summarization, and QA. Building on the GPT family of models, one of the most significant advancements in this domain is ChatGPT [13]. ChatGPT is a conversational agent driven by Generative Artificial Intelligence (GAI), developed by OpenAI and made publicly available in 2022.

ChatGPT functions as an interactive chatbot capable of replying in a manner that closely resembles human dialogue. Users can iteratively refine their prompts and guide the system's behavior. ChatGPT allows users to specify and adjust various aspects of the output, such as the desired response length and formatting style. Through these capabilities, ChatGPT provides a flexible platform for a range of applications, including

²State of Art - SQuAD, <https://paperswithcode.com/sota/question-answering-on-squad11>, 2024

educational assistance, content creation, programming help, and general information retrieval.

[4] is an important QA study in Turkish. They fine-tuned the BERT model on different QA datasets, also the study has used different language models BERTurk³ and custom LLMs. BERTurk is one of the research projects conducted to develop a large language model for the Turkish based on BERT. In our research, BERTurk served as a large language model. They achieved EM: 0.55 and F1 score: 0.67 on a limited SQuAD dataset. Another recent study [6] has developed models in the THQuAD dataset. They fine-tuned some neural network models and their results are Electra EM: 0.63 F1 score: 0.81, BERT EM: 0.62 F1 score: 0.81, Albert EM: 0.48 F1 score: 0.69. In addition, a rule-based system for question analysis in Turkish [14] concentrates on dissecting the question components of the question-and-answer system. After evaluating a question, it generates focused information derived from that question. The identified focal information can then be used to search for answers across various data sources. Generalizing rule-based studies and feature-extraction methods requires substantial human effort and language-specific investigation.

The SQuAD dataset was also evaluated in knowledge base systems, in the study of [15]. They investigated the performance of transformer models pre-trained on the SQuAD dataset and evaluated a dataset created from the knowledge base system along with additional information. The study demonstrates substantial improvement over the results of the standard models. However, the exact-match accuracy was less than 0.37. [16] A Turkish QA system was developed, enhanced with a knowledge graph, by scraping data from beyazperde.com to respond to inquiries in the movie sector. They evaluated their proposed QA pipeline and found that the question-embedding module had a minimal effect on overall performance. Although BiLSTM offers a better representation for 2-hop

³BERTurk, <https://github.com/stefan-it/turkish-bert>, 2024

questions, the SBERT model outperforms it for 3-hop questions. [17] The paper proposes a framework for answering complex questions through question decomposition and web interaction. They released a new dataset, ComplexWebQuestions, featuring questions, Protocol and RDF Query Language (SPARQL) queries, answers, and web snippets collected using their model. [18] Another study demonstrated that a universal schema serves as a more effective knowledge resource for QA than relying solely on a knowledge base or text. Their findings indicate that while a knowledge base is favored over text when it includes relevant facts, a significant number of queries still refer to text, suggesting that a combination of both text and the knowledge base outperforms the knowledge base on its own.

[19] improved short-text classification in Arabic by incorporating external knowledge, such as Wikipedia, to mitigate data sparsity and ambiguity. Semantically enriching the data on knowledge base and also using these data in the attention mechanism with deep learning that have had a positive effect on text classification in the Arabic language. [20] proposed QA method that integrates image-derived semantic representations with external knowledge base information to address questions beyond the visible content of an image. By combining these textual representations and inputting them into a recurrent neural network (RNN) alongside the question, the model achieves a more comprehensive scene understanding. This approach enables the answering of complex, natural language questions that require external context.

The knowledge base in our study was created using large language models. [21] study supported the approach by showing that large language models can keep and present many connections within the knowledge base. This study investigates the extent of relational knowledge encoded in pretrained language models without fine-tuning. The analysis reveals that models like BERT can effectively store and recall factual information, performing competitively with traditional NLP methods and supervised baselines in tasks

such as cloze-style queries and open-domain QA. [22] study that enhances knowledge base QA by incorporating external textual resources. It demonstrates substantial performance gains on the WebQuestions dataset, underscoring the value of combining unstructured text with structured knowledge base data in QA systems. [23] proposed the integration of structured knowledge bases with unstructured textual data to improve the performance of open-domain QA within the IBM Watson framework. Structured sources, such as ontologies and curated databases, provide precise and relational representations of factual knowledge, while unstructured sources, including articles, web content, and natural language documents, offer broader contextual information and linguistic diversity. By combining these complementary sources of knowledge, their approach improves Watson’s ability to interpret complex queries, retrieve relevant evidence, and generate accurate context-based answers. [24] paper proposed a multicolumn convolutional neural network approach to answer natural language questions over a knowledge base. The model captures question semantics from three perspectives: answer path, answer context, and answer type, while jointly learning low-dimensional embeddings of entities and relations. It was trained using QA pairs and enhanced through multitask learning with question paraphrases.

3. INFRASTRUCTURE AND FEASIBILITY STUDIES

3.1 The Evolution of AI in NLP

AI enables machines to interpret and produce human language, underpinning technologies such as chatbots, virtual assistants, machine translation, and automated text generation. While early approaches depended on manually crafted linguistic rules, contemporary methods. Rule-based approaches dominated early NLP research. These approaches relied on manually designed grammatical rules and expert-curated vocabulary lists. They worked only in narrow, predefined domains and broke down when faced with novel or ambiguous language. Rule-based systems were expensive to maintain and incapable of learning from new data. They required continuous manual updates. These limitations led to the development of data-driven approaches. NLP adopted statistical and probabilistic methods. The expansion of the internet and the availability of large text corpora enabled training models on real-world language data. Approaches such as n-gram models, Hidden Markov Models, and Naïve Bayes classifiers enhanced part-of-speech tagging, text classification, and machine translation.

Deep learning marked a significant turning point. RNN, LSTM, and GRUs have improved the modeling of sequential data. In parallel, word embedding techniques such as Word2Vec, GloVe, and FastText encoded words as dense vectors that capture semantic and syntactic structure. These developments significantly advanced applications such as sentiment analysis, machine translation, and NER.

In 2017, Vaswani et al. [9] proposed the Transformer architecture and self-attention. This design allows parallel sequence processing of long-range contextual dependencies. Transformer-derived models, including BERT, GPT, RoBERTa, T5, and XLNet, have achieved state-of-the-art performance on numerous NLP benchmarks.

These models use large language models trained on massive text corpora to perform a wide range of NLP tasks. LLMs such as GPT-3, GPT-4 contain billions of parameters and are trained on diverse, large-scale datasets. These models can perform multiple tasks without explicit task-specific training. Their capabilities have led to widespread adoption across education, healthcare, finance, and software development. Some use-cases are given below;

- Question answering
- Text summarization
- Code generation
- Language translation

3.2 Evaluation and Selection of Dataset

3.2.1 *Evaluation Methodology and Datasets*

High-quality datasets are essential for training QA systems. However, nearly all existing datasets are in English. The Turkish language does not have as rich a dataset as English. Because building such a dataset from scratch is extensive and expensive, the chosen approach is to translate a selected existing dataset into Turkish. Numerous datasets were reviewed, and the following objectives guided the selection process.

- Containing detailed/superficial, long/short questions, texts, and different domain content.
- Given specific truth tables, determine the number of conducted studies.
- Sufficient dataset size for model training and evaluation

The following criteria have been extensively analyzed based on the above criteria: AI2 Reasoning Challenge (ARC) [25], Large-scale Reading Comprehension Dataset From Examination (RACE) [26], MCTest [27], the SQuAD [1], Story Cloze Test [27], and Facebook bAbI [28].

ARC is a benchmark designed to evaluate deep reasoning and scientific understanding at roughly the level of an elementary school student. It consists of multiple-choice questions divided into two subsets: ARC-Easy and ARC-Challenge, designed to assess multi-step reasoning and the integration of prior scientific knowledge. Main characteristics:

- Concentrates on science-related questions
- Uses 4-option multiple-choice items
- Demands both external knowledge and inference
- The Challenge subset is constructed to avoid straightforward lookup

Large-scale Reading Comprehension Dataset From Examination (RACE) is derived from English exams administered to Chinese middle and high school students. Its primary aim is to assess deep reading comprehension in realistic test settings. Main characteristics:

- 100K+ questions
- Multiple-choice exam format
- Focus on understanding, reasoning, and inference
- Includes long reading passages

MCTest is a collection of stories for children with multiple-choice questions. The dataset was designed to assess paragraph-level comprehension. Main characteristics:

- Contains two parts: MC160 and MC500
- Created by short passages
- Require inferring the correct answer rather than extracting it verbatim

SQuAD is a reading comprehension dataset derived from Wikipedia articles. It was designed to evaluate the extractive understanding of natural text. Main characteristics:

- Includes 100,000+ question–answer pairs
- Answers are continuous text spans within the passage
- SQuAD 2.0 has questions with no valid answer

The Story Cloze Test presents a brief narrative in which the last sentence is omitted. The system must choose from two candidate endings. Its goal is to assess comprehension and commonsense inference. Main characteristics:

- Evaluates understanding of narratives
- Measures reasoning and coherence
- Binary choice between two possible answers

The Facebook bAbI dataset consists of synthetic tasks. It was designed on 20 distinct reasoning tasks. Main characteristics:

- Synthetic benchmark tasks
- Includes spatial, temporal, and logical inference
- Limited lexicon and grammar

3.2.2 Comparison between Datasets

[29], [30] provide some specific criteria to evaluate general requirements. Table 3.1 presents a structured comparison of six MRC datasets: ARC, RACE, MCTest, SQuAD, Story Cloze, and Facebook bAbI. Basic evaluation metrics are realism, reasoning complexity, linguistic diversity, scale, knowledge requirement, and usefulness for LLMs. These comparison studies show the strengths and limitations of datasets.

- Realism: Natural, real-world language
- Reasoning Complexity: Depth of inference
- Linguistic Diversity: Vocabulary richness
- Dataset Size: Size for deep learning training
- Knowledge Requirement: Needed domain knowledge
- LLM Suitability: Usability for large model training

Table 3.1: Benchmark comparison of MRC datasets

Dataset	Realism	Reasoning	Diversity	Size	Knowledge	LLM Usability
AI2-ARC	Medium	Very High	Medium	Medium	High	Medium
RACE	High	High	High	Large	Medium	High
MCTest	Medium	Medium	Low	Small	Low	Low
SQuAD	Very High	Medium	Very High	Large	Medium	Very High
Story Cloze	Medium	High	Medium	Medium	Medium	Medium
Facebook bAbI	Low	High (Logical)	Low	Small	None	Low

In terms of realism, SQuAD and RACE stand out as the most representative of real-world language use. SQuAD was derived from Wikipedia articles, encyclopedic writing, and information structures. This feature is valuable for MRC tasks. Similarly, RACE contains examination passages written for real students. By contrast, Facebook bAbI shows the lowest realism score because it is synthetic. This feature helps test precise reasoning abilities. But it lacks the complexity of natural language. AI2-ARC, MCTest, and

Story Cloze fall in the middle, as they incorporate realistic themes but are still constrained by specific structural or stylistic limitations.

Regarding reasoning complexity, AI2-ARC and Facebook bAbI exhibit very high logical and multi-step inference demands. AI2-ARC’s challenge subset is explicitly constructed to avoid simple textual matching and instead requires multi-hop reasoning. Similarly, bAbI tasks focus on logical, spatial, and temporal logic. RACE and Story Cloze also demonstrate high reasoning requirements, as they often require interpretation of intent, context, and implicit meaning. In contrast, SQuAD and MCTest involve medium-level reasoning, as many questions can be answered through direct span extraction.

The dimension of linguistic diversity favors datasets with large, human-generated sources. SQuAD and RACE have wide-ranging vocabulary, varied syntactic constructions, and diverse topics. These properties make them particularly effective for training general-purpose language models. MCTest and Facebook bAbI have lower diversity due to their simple sentence structures and limited vocabularies. This constrains their ability to challenge modern deep learning models. Story Cloze and AI2-ARC maintain moderate diversity, but they have narrower topic and style boundaries.

In terms of dataset size, SQuAD and RACE are large, with over 100,000 questions each. This is essential for training transformer-based architectures. AI2-ARC and Story Cloze are medium-sized. MCTest and bAbI remain small-scale datasets, generally used for controlled experiments, evaluations, or proof-of-concept studies rather than full model training.

AI2-ARC relies heavily on scientific knowledge. RACE and SQuAD require world knowledge but rely on information present in the passages. Story Cloze depends more on common sense and narrative understanding than on factual knowledge. MCTest and Facebook bAbI don’t require external knowledge, as all necessary information is contained within the given text in a simplified format.

Usability for LLM strongly correlates with dataset size, diversity, and realism. SQuAD and RACE receive the highest rating in this category due to their large scale and high linguistic diversity. AI2-ARC and Story Cloze are considered moderately useful, often employed in evaluation settings rather than full-scale training. In contrast, Facebook bAbI and MCTest are less suitable for LLM training due to their small size and limited linguistic complexity, but they are valuable for some experiments.

Overall, no single dataset is sufficient to fully evaluate a model’s reading comprehension and reasoning abilities. SQuAD and RACE are more effective for assessing real-world language understanding. AI2-ARC and bAbI are better suited for testing reasoning and logic. Story Cloze fills an important role in evaluating narrative and commonsense reasoning. But we chose the SQuAD dataset to create more comparative analysis and evaluate large language models under realistic linguistic conditions.

3.3 Determining the Model Methodology & Fail Cases

Attention-based and transformer-based architectures in NLP have achieved state-of-the-art results. However, initial investigations have explored the impact of various methods using core NLP-specific techniques. In this section, we present the results of models and methods evaluated without transformer-based training or large language models.

3.3.1 *Dependency Parser Similarity*

Dependency parsing is a method in NLP that analyzes a sentence’s grammar by detecting syntactic relations between words in a sentence. It represents syntax directly as head–dependent word pairs, rather than segmenting the sentence into phrase structures (e.g., noun or verb phrases). The result of this process created a dependency tree, in which the main verb of the sentence is typically the root, and all other words are connected to it, either directly or indirectly. Each word in a sentence is associated with:

- Head: the word on which it depends
- Dependency label: the type of syntactic relation (e.g., *nsubj*, *obj*, *amod*)

"The child reads a book."

The dependency relationships can be expressed as follows:

- **reads** → ROOT
- **child** → *nsubj* of *reads*
- **book** → *obj* of *reads*
- **The, a** → determiners (*det*)

This structure represents the grammatical dependencies between words and helps identify semantic roles such as the subject and the object of the sentence. Stanza Library¹ is an open-source NLP library developed by the *Stanford NLP Group*. Stanza is based on modern deep learning architectures such as BiLSTM and Transformer-based models, and it supports more than 60 languages, including English, Turkish, Spanish, and Chinese. It is widely used in both academic research and industrial applications. It provides state-of-the-art neural network models for a variety of NLP tasks, including:

- Tokenization
- Part-of-Speech (POS) tagging
- Lemmatization
- NER

¹Stanza Library: <https://stanfordnlp.github.io/stanza/>

- Dependency Parsing

Studies have been conducted on using dependency parsing to find the sentence closest to the answer within the question and its context in QA systems. The question and the sentences within the content were separated into dependency parser components, and an approach was developed based on their similarities. The sentences were searched for the most similar dependency components to the question. The results are presented in the Table 3.2. The table presents the performance of a dependency parser-based sentence similarity approach in identifying sentences that contain the correct answer. For example, a question is broken down into dependency parsing components as follows. This is applied to all sentences in the content. Similarity is searched between the question and the sentences.

- **Input:** Super Bowl 50 nerede gerçekleşti?
- **Output:** # Super nsubj gerçekleşti # Bowl flat Super # 50 nsubj gerçekleşti # nerede obl gerçekleşti # gerçekleşti root ? # ? punct gerçekleşti

Table 3.2: *Dependency parser similarity results*

Dependency Parser Similarity	Count	Sentence Contains Answer (%)
Count The Most Similar Sentences Contains Answer	7513	30%
Count Greater Than Threshold	17757	71%
Count The Less Similar Contains Answer	1811	7%
Count Less Than Threshold	7026	28%

Most Similar Sentences Containing the Answer (7,513 – 30%): This indicates that in 30% of the cases, the top-ranked (most similar) sentence found by the dependency parser actually contained the correct answer. This reflects the direct effectiveness of the similarity ranking.

Greater Than Threshold (17,757 – 71%): We consider all sentences whose similarity score is above a predefined threshold, the correct answer appears in 71% of them. We set a threshold for similarity as 0.5 (%50).

Less Similar but Still Containing the Answer (1,811 – 7%): These are sentences that are not very similar syntactically, but still include the correct answer. This is important because it shows that syntactic similarity alone is not always sufficient to guarantee correct answer detection.

Less Than Threshold (7,026 – 28%): Sentences with similarity scores below the threshold did not contain the answer.

The dependency parser provides some data on similarities, but it is not a sufficient end-to-end solution for accurately identifying the answer.

3.3.2 *NER Similarity*

NER is one of the fundamental tasks in NLP that focuses on identifying and classifying named entities in a given text into predefined semantic categories. These categories usually include persons, organizations, locations, dates, times, monetary values, percentages, products, and events. After applying NER, the following entities can be extracted:

"Lebron James was born in Ohio."

- **Lebron James** → PERSON
- **Ohio** → LOCATION

Studies have been conducted on using NER similarity to find the sentence closest to the answer within the question and its context in QA systems. The question and the sentences within the content were separated into NER components, and an approach was developed based on their similarities. The sentences were searched for the most similar NER to the question. The results are presented in the Table 3.3. The table presents the performance of NER similarity approach in identifying sentences that contain the correct answer. For example, a question is broken down into NER components as follows. This

is applied to all sentences in the content. Similarity is searched between the question and the sentences.

Table 3.3: *NER-based similarity results*

NER Similarity	Count	Sentence Contains Answer (%)
Count The Most Similar Sentences Contains Answer	4346	17%
Count Greater Than Threshold	7781	31%
Count The Less Similar Contains Answer	4980	19%
Count Less Than Threshold	17002	68%

Most Similar Sentences Containing the Answer (4,346 – 17%): Sentences selected by the NER-based similarity method actually contain the correct answer. This shows that NER similarity alone is relatively weak when only the most similar sentence is considered.

Greater Than Threshold (7,781 – 31%): When all sentences with a NER similarity score above a defined threshold are included (above 0.5 - %50), the answer appears in 31% of them. This is an improvement compared to using only the top sentence, but still considerably lower than dependency-based similarity (which reached 71%).

Less Similar but Containing the Answer (4,980 – 19%): These sentences have low NER similarity, but still contain the correct answer in 19% of cases. This clearly indicates that NER is not the primary determinant of answer relevance.

Less Than Threshold (17,002 – 68%): The sentences that fall under the threshold do not contain the answer. This proves that the NER similarity method is not very successful in finding the answer.

3.3.3 *Semantic Similarity*

Cosine similarity is a widely used metric for measuring the semantic similarity between two text sequences in NLP. When combined with large language models such as BERT, cosine similarity becomes a powerful method for evaluating the semantic closeness between sentences.

BERT provides a transformer-based language model infrastructure that generates

contextualized representations for words and sentences. Given an input sentence, BERT converts it into a dense, high-dimensional vector that captures its semantic meaning within context. BERT produces contextual word embeddings. *Sentence-BERT (SBERT)* is a fine-tuned version of BERT, specifically optimized for generating semantically meaningful sentence embeddings. Therefore, the combination of SBERT and cosine similarity has become a standard approach for semantic similarity.

The question and the sentences in the content were split into semantic vector representations using the upcoming SBERT. The sentences were searched for the most semantically similar to the question. The results are presented in the Table 3.4. The table presents the performance of the semantic similarity approach in identifying sentences that contain the correct answer. Similarity between the question and the sentences is measured using cosine similarity.

Table 3.4: *Semantic similarity results*

Semantic Similarity	Count	Sentence Contains Answer (%)
Count The Most Similar Sentences Contains Answer	3920	15%
Count Greater Than Threshold	6403	25%
Count The Less Similar Contains Answer	5404	21%
Count Less Than Threshold	18380	74%

Most Similar Sentences Containing the Answer (3,920 – 15%): The sentences were ranked as the most semantically similar that contain the correct answer. Its performance is quite low. We observed that the semantic similarity does not necessarily imply answer relevance in a MRC task.

Greater Than Threshold (6,403 – 25%): All sentences above the similarity threshold, the answer appears in 25% of them. This is slightly better than the “most similar” subset, but still insufficient for reliable answer extraction on its own.

Less Similar but Containing the Answer (5,404 – 21%): This part shows that answer-bearing sentences are not always semantically close to the question, at least not in an embedding space.

Less Than Threshold (18,380 – 74%): Sentences below the threshold contain the answer. This indicates that the threshold is ineffective at filtering sentences.

The results demonstrate that semantic similarity alone is not sufficient to identify answer-containing sentences accurately. These findings highlight the need to integrate semantic similarity with syntactic methods, such as dependency parsing and structural analysis.

3.3.4 Morphological Analysis & Data Summarization

A morphological analyzer is an NLP component that decomposes words into their smallest meaningful units (morphemes). It also assigns grammatical features to them. These grammatical features include the word stem (root), affixes (prefixes, suffixes, and infixes), and morphological attributes such as part-of-speech, tense, number, case, person, and gender. The output of a morphological analyzer is often used for lemmatization, part-of-speech tagging, or semantic processing.

A sample word in Turkish: *evlerimizden* ("from our houses"), morphological segmentation: *ev + ler + imiz + den*

- Stem: *ev* (house)
- Suffixes: *-ler* (plural), *-imiz* (1st person plural possessive), *-den* (ablative case: from)
- Features: {POS=NOUN, Number=Plur, Poss=1Pl, Case=Abl}

The system's data summarization efficiency was tested by removing morphological components that did not change the meaning of the content data. Under the Include title, the elements in the data remain. Under the exclude title, the morphological components are removed from the content data.

- **Included :**

- Noun
- Verb
- Conjunctions
- Prepositions

- **Excluded:**

- Determiners
- Pronouns
- Adjectives
- Adverbs

Morphological analyzers are a foundational component in NLP pipelines, especially for agglutinative and highly inflected languages. Summarization data negatively affects the performance of the currently developed BERT-base model (F1: -5%), indicating that this method does not improve overall performance. Since the approaches mentioned above, such as Dependency Parsing, NER, and data summarization, did not improve the model's performance, we continued the transformer-based architecture to find an optimum solution.

3.4 End-to-end Pre-Studies & Trials

3.4.1 Creation Pre-Study Dataset

In the study, a demo study was conducted to develop a deep learning model by translating an existing dataset into Turkish. We aimed to create a roadmap without undertaking a study. The pre-study aims to translate a portion of a light dataset into Turkish

using automatic translation, then continue working with the SQuAD dataset using a more efficient model by evaluating different deep learning baselines.

As a result of all the investigations, Facebook bAbI has been decided as the most suitable dataset for this study. The bAbI dataset contains many tasks. Each task simulates a different inference task. The task list is shown in the Table 3.5. The bAbI dataset is synthetic. It is generated by software, with short sentence structure and minimal vocabulary. It will be more illustrative to give examples of the data in Figure 3.1. The red-colored parts indicate the expected answers from the dialogues. Tasks in the dataset have 10K or 1K train scenarios. The dataset supports multiple languages, including English and Hindi. Its data structure is represented in Table 3.6.

Task 1: Single Supporting Fact Mary went to the bathroom. John moved to the hallway. Mary travelled to the office. Where is Mary? A:office	Task 2: Two Supporting Facts John is in the playground. John picked up the football. Bob went to the kitchen. Where is the football? A:playground
Task 3: Three Supporting Facts John picked up the apple. John went to the office. John went to the kitchen. John dropped the apple. Where was the apple before the kitchen? A:office	Task 4: Two Argument Relations The office is north of the bedroom. The bedroom is north of the bathroom. The kitchen is west of the garden. What is north of the bedroom? A: office What is the bedroom north of? A: bathroom
Task 5: Three Argument Relations Mary gave the cake to Fred. Fred gave the cake to Bill. Jeff was given the milk by Bill. Who gave the cake to Fred? A: Mary Who did Fred give the cake to? A: Bill	Task 6: Yes/No Questions John moved to the playground. Daniel went to the bathroom. John went back to the hallway. Is John in the playground? A:no Is Daniel in the bathroom? A:yes
Task 7: Counting Daniel picked up the football. Daniel dropped the football. Daniel got the milk. Daniel took the apple. How many objects is Daniel holding? A: two	Task 8: Lists/Sets Daniel picks up the football. Daniel drops the newspaper. Daniel picks up the milk. John took the apple. What is Daniel holding? milk, football
Task 9: Simple Negation Sandra travelled to the office. Fred is no longer in the office. Is Fred in the office? A:no Is Sandra in the office? A:yes	Task 10: Indefinite Knowledge John is either in the classroom or the playground. Sandra is in the garden. Is John in the classroom? A:maybe Is John in the office? A:no

Figure 3.1: Facebook bAbI examples

Table 3.5: Facebook bAbI project question and answer task list

#	Task Definition
1.	Single Supporting Fact
2.	Two Supporting Facts
3.	Three Supporting Facts
4.	Two Argument Relations
5.	Three Argument Relation
6.	Yes/No Questions
7.	Counting
8.	Lists/Sets
9.	Simple Negation
10.	Indefinite Knowledge
11.	Basic Co-reference
12.	Conjunction
13.	Compound Co-reference
14.	Time Reasoning
15.	Basic Deduction
16.	Basic Induction
17.	Positional Reasoning
18.	Size Reasoning
19.	Path Finding
20.	Agent's Motivation

Table 3.6: Facebook bAbI project QA task data format

ID text	1 Mary moved to the bathroom.
ID text	2 John went to the hallway.
ID question[tab]answer[tab]supporting fact IDS.	3 Where is Mary? bathroom 1

It has 20 different tasks for this dataset; the difficulty of these tasks varies according to their levels. The first three Facebook bAbI QA tasks were translated into Turkish using Google Translate. The size of these three tasks is given below.

- Single Supporting Fact (Simple), 6.000 line
- Two Supporting Facts (Medium), 12.000 line
- Three Supporting Facts (Hard), 60.000 line

3.4.2 Assessment of Deep Learning Approaches

Deep learning in NLP refers to the use of multi-layer neural network models to automatically learn linguistic patterns, structures, and semantics from large amounts of text data. Unlike traditional NLP methods that relied heavily on hand-crafted features (such as grammar rules, POS tags, or manually engineered n-grams), deep learning enables models to learn representations directly from raw text. Deep learning in NLP is a subfield of AI in which deep neural networks such as RNNs, LSTMs, GRUs, Transformers, and large language models are used to understand, process, and generate human language.

RNN [31] was designed to process sequential data. RNNs maintain a hidden state vector that is updated at each time step, enabling the model to capture dependencies. However, RNNs struggle with long-term dependencies due to the vanishing and exploding gradient problems. The lack of capability limits their ability to retain information over long sequences.

RNNs [32], [33] are a class of neural network architectures specifically designed to model sequential and temporal data by maintaining an internal memory of previous inputs. Unlike feed-forward neural networks, RNNs introduce recurrent connections that allow information to persist across time steps, enabling the network to capture dependencies within sequences such as text, speech, and time-series data. At each time step, the hidden state of an RNN is updated based on the current input and the previous hidden state, typically through nonlinear transformations, which allows the model to learn contextual representations over sequences. Despite their conceptual simplicity and effectiveness in modeling short-term dependencies, standard RNNs suffer from critical limitations, most notably the vanishing and exploding gradient problems during backpropagation through time, which hinder their ability to learn long-range dependencies. This weakness reduces performance on tasks requiring long contextual understanding. However, RNNs remain

powerful for sequence modeling due to parameter sharing across time steps, computational efficiency, and their foundational role in the development of more advanced architectures such as Long Short-Term Memory and Gated Recurrent Unit networks, which explicitly address these limitations. As a result, RNNs have played a crucial role in the evolution of deep learning approaches for natural language processing and sequential data analysis [34].

Long Short-Term Memory (LSTM) [35] extended RNNs by introducing memory cells and gating mechanisms. LSTM achieves this by introducing a memory cell and a set of gating mechanisms, namely the input gate, forget gate, and output gate, which regulate the flow of information into, within, and out of the cell state. The forget gate determines which information should be discarded from the previous cell state, the input gate controls which new information should be stored, and the output gate decides which information is exposed to the next hidden state. This gated architecture enables [36] [34] LSTM networks to retain relevant contextual information over long sequences, making them particularly effective in tasks such as language modeling, machine translation, speech recognition, and time-series forecasting. Despite their powerful capability to capture long-range dependencies, LSTMs suffer from certain limitations, including high computational complexity, slower training due to sequential processing, and difficulty in parallelization compared to transformer-based models. Additionally, LSTMs require careful hyperparameter tuning and large datasets to achieve optimal performance. Nevertheless, their robustness in handling long-term dependencies and resistance to gradient degradation make LSTMs a foundational and influential architecture in deep learning for sequential data processing.

Gated Recurrent Units (GRUs) [37] [38] is a variant of LSTMs and combines the cell state and hidden state into a single vector. GRUs use two gates: the update gate and the reset gate. The update gate controls the extent to which the past information is

preserved. The reset gate determines how the previous state is combined with the new input. GRUs achieve comparable performance to LSTMs, but it requires fewer parameters and faster training. By adaptively combining the previous hidden state with the current input, GRUs are capable of capturing long-term dependencies in sequential data more effectively than traditional RNNs. One of the main strengths of GRU is its relatively simple architecture, which requires fewer parameters and leads to faster training and lower computational cost compared to LSTM, making it suitable for resource-constrained environments. However, the reduced number of gates may limit its ability to model highly complex temporal patterns, and in some tasks, LSTM models can outperform GRUs due to their more expressive memory control. Despite this limitation, GRUs have demonstrated strong performance in various natural language processing tasks such as machine translation, speech recognition, and text classification.

BERT [10] uses only the encoder portion of the Transformer architecture. BERT employs self-attention mechanisms to capture contextual relationships between tokens. BERT achieves state-of-the-art performance across numerous NLP tasks, classification, QA, and semantic similarity [39]. Unlike traditional unidirectional language models, BERT processes the entire input sequence simultaneously using a multi-layer Transformer encoder architecture based on self-attention mechanisms, allowing each token to attend to both its left and right context. BERT is pre-trained using two unsupervised objectives: Masked Language Modeling, where a subset of input tokens is randomly masked and predicted, and Next Sentence Prediction, which enables the model to capture inter-sentence relationships. This bidirectional pre-training enables BERT to achieve state-of-the-art performance on a wide range of natural language processing tasks such as QA, NER, and text classification after fine-tuning. The primary strengths of BERT include its ability to model rich contextual dependencies, scalability to large datasets, and strong transfer learning capabilities across tasks. However, BERT also has notable limitations, including

high computational and memory costs, slower inference compared to recurrent models, and a fixed input length constraint due to the self-attention mechanism. Despite these weaknesses, BERT has become a foundational model in modern NLP research and has inspired numerous extensions and improvements.

In our trials, Table 3.7 provides an overview of four neural network architectures in NLP: RNN, LSTM, GRU, and BERT. The comparison focuses on characteristics such as architecture, ability to long-term dependencies, parallelization, and training speed. RNNs are suitable for simple recurrence and capable of modeling sequences. However, they are ineffective at capturing distant dependencies due to vanishing gradients. LSTMs and GRUs address long-term dependencies through gating mechanisms. LSTMs use input, forget, and output gates, and GRUs use update and reset gates. This infrastructure allows both models to retain information over longer spans. However, neither of them can be parallelized during training.

BERT is based on the Transformer encoder architecture and uses self-attention to capture global dependencies across an entire sequence. This enables full parallelization, improving efficiency and performance. As a result, BERT demonstrates excellent long-term dependency handling and faster training than recurrent models.

Table 3.7: *Comparison of RNN, LSTM, GRU, and BERT models*

Feature	RNN	LSTM	GRU	BERT
Architecture Type	Recurrent	Recurrent w/ gates	Recurrent w/ gates	Transformer Encoder
Long-Term Dependency Handling	Poor	Good	Good	Excellent
Parallelization	No (sequential)	No	No	Yes (self-attention)
Training Speed	Slow	Slower	Faster than LSTM	Fast (parallelizable)

3.4.3 Evaluation Metrics

We measured the models' success using Exact Match (EM) and F1 score. I will explain how these scores are calculated with some examples below. **EM** measures the strictest form of correctness in question answering tasks. EM evaluates whether the predicted answer exactly matches the ground-truth answer after applying a normalization process. The normalization typically includes:

- Converting all characters to lowercase,
- Removing punctuation,
- Removing suffixes,
- Removing extra whitespace

Example Ground Truth Answer: Jane Austen

Table 3.8: *Exact Match (EM) evaluation examples*

Model Prediction	EM
Jane Austen	1
Austen	0
Jane	0
Jane Austen (1775–1817)	0

F1-score measures partial correctness by evaluating the overlap between the predicted answer and the ground-truth answer. F1-score is computed at the token level by:

1. Tokenizing both the predicted answer and the ground-truth answer,
2. Calculating Precision and Recall:

$$\text{Precision} = \frac{\text{Number of overlapping tokens}}{\text{Number of predicted tokens}}$$

$$\text{Recall} = \frac{\text{Number of overlapping tokens}}{\text{Number of ground-truth tokens}}$$

The F1-score is then calculated as the harmonic mean of Precision and Recall:

$$\text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Example: Ground Truth Answer: Jane Austen

Table 3.9: *F1-score evaluation examples*

Prediction	Precision	Recall	F1
Jane Austen	1.0	1.0	1.0
Jane	1.0	0.5	0.67
Austen	1.0	0.5	0.67
Jane Austen wrote it	0.5	1.0	0.67

3.4.4 Pre-evaluation Deep Learning Approaches

Table 3.10 presents the performance of RNN, LSTM, GRU, and BERT on the first three tasks of the bAbI dataset in Turkish, which evaluate reasoning over one, two, or three supporting facts. In the first task, which requires identifying a single supporting fact, all three recurrent models (RNN, LSTM, and GRU) achieve similar accuracy around 0.5, while BERT significantly outperforms them with 0.77. This performance gap increases in the second task, where the models must integrate two supporting facts. Here, RNN, LSTM, and GRU show lower accuracy. BERT, however, maintains strong performance at 0.68, demonstrating its superior capability for long-dependent reasoning. The third task requires combining three supporting facts. Recurrent models around 0.20 accuracy, while BERT achieves 0.63. These results highlight that recurrent architectures struggle with long-range reasoning and the integration of multiple facts. BERT’s Transformer-based attention mechanisms enable it to perform substantially better on complex reasoning tasks.

Table 3.10: *EM Performance of RNN, LSTM, GRU, BERT for the first three bAbI tasks*

Task Number	RNN	LSTM	GRU	BERT
QA1 - Single Supporting Fact	0.53	0.50	0.53	0.77
QA2 - Two Supporting Facts	0.27	0.20	0.30	0.68
QA3 - Three Supporting Facts	0.20	0.21	0.20	0.63

4. CREATING DATASET

4.1 SQuAD Dataset

SQuAD is a large-scale benchmark for machine reading comprehension that is organized as triples of content passages (c), questions (q), and corresponding answers (a). Each data point corresponds to a passage from a Wikipedia article and includes human-generated questions and answers within that passage. To construct this dataset, the creators gathered 10,000 diverse Wikipedia articles, spanning a broad range of topics.

SQuAD is divided into two main subsets: a training set and a development set. The training set contains approximately 90% of the total articles, providing substantial data for models to learn patterns of QA behavior. The development set accounts for about 10% of the articles and is used to tune model hyperparameters and evaluate performance during development. This split is designed to promote robust generalization and fair comparison among different systems. In comparison with earlier or smaller-scale machine reading comprehension datasets, SQuAD stands out in both size and difficulty. SQuAD contains a vast number of questions. The questions require the model to carefully read and understand a single passage to find the answer span. These questions are often formulated in a way that discourages simple keyword matching and instead requires deeper comprehension, co-reference resolution, and sometimes multi-sentence reasoning. As a result, the dataset poses challenges for NLP systems.

Comprehensive statistics about the dataset, such as the distribution of questions, passages, and answers in Tables 4.1 and 4.2. These tables help illustrate the diversity and complexity of SQuAD and serve as a useful reference for researchers.

Table 4.1: *The statistics of SQuAD*

Statistic Items	Number-of-Items and Value
Question	Train:87599 Development:10570
Paragraph	Train:18896 Development:2067
Title	Train:442 Development:48
Human EM Score	86.831
Human F1 Score	89.452
IE-Net EM (Sota) Score	90.939
IE-Net F1 (Sota) Score	93.214

Table 4.2: *The sample data of SQuAD*

Item	Value
Title	Nikola Tesla
Paragraph	In 1870, Tesla moved to Karlovac, to attend school at the Higher Real Gymnasium, where he was profoundly influenced by a math teacher Martin Sekuli. The classes were held in German, as it was a school within the Austro-Hungarian Military Frontier...
Question	Who was Tesla influenced by while in school?
Answer1	Martin Sekuli
Answer2	Martin Sekuli
Answer3	math teacher Martin Sekuli
Answer Start Point	148

4.2 The Creation of mSQuAD

The training and development splits of the SQuAD dataset were first translated into Turkish using Microsoft Translator, and the resulting Turkish version was made publicly available on GitHub [40]. The entire translation procedure, which involved sending the original English content to the API and collecting the Turkish outputs, lasted approximately 3 days from start to finish. Microsoft Translator uses a character-based pricing model, charging for each request based on the number of characters processed. The total cost of this operation was around \$400.

After the initial translation was completed, we analyzed how many of the translated samples still preserved the original answer span within the translated context. In other words, we measured the proportion of instances for which the “content contains answer” condition continued to hold after translation. This rate was 62%. A value at this level indicates that, in roughly 4 out of 10 samples, the answer text is no longer correctly aligned or even present in the translated context. This implies that close to half of the dataset is potentially unusable for extractive QA tasks in its current form. Misalignment issues can arise.

In addition to this Microsoft-based Turkish SQuAD translation, there exists another partial Turkish version of SQuAD known as gSQuAD, which was produced by translating the data using Google Translate. This alternative dataset covers approximately 50% of the original SQuAD samples [4]. We incorporated gSQuAD into our study as a comparative benchmark to better understand differences in translation quality. We measured data coverage and its downstream impact on model performance.

As we explored how data quantity influences model performance, we applied a series of targeted corrections and post-processing steps to our translated data. We aimed to maximize the number of items in which the answer span is preserved in the Turkish context. This process also allowed us to carefully analyze how translation errors propagate

through the training data and examine how improvements in translation can be made.

For this purpose, we separately considered and processed the three core components of each SQuAD instance: the content paragraph (c), the question (q), and the answer span (a). These elements were translated in batches using Microsoft Translator, which facilitated both automated processing and subsequent manual or semi-automatic error checking. After obtaining the initial batch translations, we performed multiple rounds of error identification and correction. Typical issues included incorrect sentences and mis-translation of domain-specific terms. Inconsistencies between the wording of the question and the phrasing of the answer span the context. Named entities were translated in a way that no longer matched the context.

Through this iterative refinement process, we substantially increased the proportion of samples that satisfied the “content contains answer” requirement. As a result, the usable portion of the dataset rose from 62% to 78%. This improvement means that a significantly larger share of the translated training and evaluation data for extractive QA models in Turkish can be used. Illustrative examples of the final translated paragraph, the question, and the answer span are presented in Table 4.3.

4.3 Post Translation Actions

To minimize data loss and maximize the size and quality of the final dataset, we systematically examined the error cases in the translated output. These error instances were analyzed to identify common patterns and correctable issues. Using this insight, we applied specially developed data-fix batch codes to the translated data. Through this process, a significant portion of previously unusable data could be recovered and incorporated into the usable dataset.

Case 1: In the first group of issues, we identified a range of basic syntax and formatting errors across several translated answers. For instance, there were unnecessary

Table 4.3: *The sample data of mSQuAD*

Item	Value
Title	Nikola Tesla
Paragraph	1870’te Tesla, Yüksek Gerçek Spor Salonu’ndaki okula gitmek için Karlovac’a taşındı ve burada matematik öğretmeni Martin Sekulić’den derinden etkilendi. Dersler Avusturya-Macaristan Askeri Sınırı içindeki bir okul olduğu için Almanca olarak yapıldı. Tesla kafasında hesap yapabildi, bu da öğretmenlerini kopya çeken olduğuna inanmaya itti. 1873’te mezun olarak üç yılda dört yıllık bir dönemi bitirdi.
Question	Tesla okuldayken kimden etkilendi?
Answer1	Martin Sekulić
Answer2	Martin Sekulić
Answer3	matematik öğretmeni Martin Sekulić
Answer Start Point	114

punctuation marks such as stray periods, commas, or extra spaces appearing at the end of some translations (e.g. en: “alternating current” tr: “alternatif akım.”). These additional characters are not present in the original text and therefore should not appear in the translated output. In addition, there were cases in which visually similar characters with different encoding values were inconsistently used in the translations. An example of this is the use of varying dash characters, such as “–” and “-”, where the content shows “14-12” but the answer contains “14-12” with a different type of hyphen or dash, leading to subtle but essential mismatches at the character encoding level.

Furthermore, we observed numerous errors related to date format translation. Instead of producing a single, correct, and consistent date, the system often generated redundant or incorrect variants. For example, the English answer “February 7, 2016” was translated into Turkish as “7 Şubat 2016, Şubat 2016, Bugün”, which not only includes the correct date but also an incomplete date (“Şubat 2016”) and an irrelevant temporal reference (“Bugün” meaning “Today”). The correct Turkish translation should be “7 Şubat

2016”. These issues indicate that the translation process sometimes introduces extra tokens, merges multiple interpretations, or mishandles date formats, leading to outputs that are syntactically incorrect, semantically confusing, or not faithful to the source text.

Case 2: In this scenario, the answers appeared in various letter cases. Our first step was to normalize all data to lowercase. This straightforward transformation, however, did not fully resolve all the matching issues we encountered. Turkish documents can include specific English terms while still following language-specific casing rules that differ from English. For example, Turkish distinguishes between I-ı, I-İ, and i-i, whereas English operates with a simpler I-i distinction. To address this, we went beyond basic case normalization and detected and replaced language-specific characters where necessary. After taking these steps, we were able to obtain EM between the context and answer spans by adjusting casing. As a final refinement, we implemented and applied custom-developed routines for both uncapitalizing and capitalizing text. This combination of normalization, character replacement, and specialized casing methods enabled us to resolve case-related mismatches robustly.

Case 3: In some instances, specific English proper nouns appearing in the answers were incorrectly translated into Turkish. Whenever this occurred, and the original English term could be clearly identified within the Turkish content, the translated answer was revised to match the English version exactly. For instance, the name of the American sports team “Arizona Cardinals” is already used as “Arizona Cardinals” in the Turkish text. Still, in the answer section, it had been translated as “Arizona Kardinalleri” In such a case, the answer was corrected to consistently use “Arizona Cardinals” aligning it with the form used in the Turkish content and with the original English source.

Furthermore, there was a normalization step involving definite articles. When comparing English answers, if they began with the word “The,” this leading “The” was

stripped from the original English answer data before performing any matching or comparison operations. This preprocessing helped avoid superficial mismatches caused solely by the presence or absence of “The” and ensured more accurate alignment between English and Turkish entries.

Together, these two types of adjustments, correcting translated proper nouns to match their original English form when present in the Turkish content, and removing the initial ‘The’ from English answers for comparison, resulted in the identification and incorporation of additional aligned items. Consequently, these refinements increased the overall volume and consistency of the dataset, enhancing its quality and usefulness for downstream tasks.

Case 4: In this case, various inconsistencies were identified between the content and the answers. These mismatches were related to monetary values, numbers, and mathematical notations. They often arose from locale-specific conventions, such as the use of commas as thousands separators, decimal separators, or from alternative ways of expressing the same currency or percentage.

For instance, the original content might display the value as “1,200”. In contrast, the translated answer could represent the same number as “1.200”. Although both notations can denote one thousand two hundred, they follow different regional formatting standards. Similarly, a percentage like “10%” in the content might appear as “%10” in the answer, reflecting a change in the placement of the percent sign rather than a change in numerical value. Another example involves currency formatting: a monetary amount such as “5,000,000\$” in the original content might be rendered as “5.000.000 dollar” in the translated answer, where the symbol “\$” is replaced by the word “dollar,” and the separator style is altered.

These kinds of inconsistencies, misalignment between context and answers, and

potential errors in downstream tasks that rely on precise numerical information. To address this issue, systematic corrections were applied to align the numerical and monetary expressions in the answers with the source content. In the batch script, problematic characters, symbols, or terms were stripped from the translation output and replaced with standardized, correct values that matched the original format. This process ensured that the numerical meaning and representation remained consistent across both content and answers.

The cumulative impact of these normalization and correction procedures on the overall dataset is summarized in Table 4.4. In addition, the table highlights the extent to which applying these corrections enhanced internal consistency and reduced discrepancies and irregularities. Together, these metrics provide a clear overview of the scope of corrections and their contribution to improving the quality of the data used in subsequent analyses.

Table 4.4: *Effect of the post-translation correction type to translated answers*

Case-Number	Devevelopment-Dataset-Rate	Train-Dataset-Rate
Case 1	+%5	+%3
Case 2	+%6	+%5
Case 3	+%4	+%4
Case 4	+%1	+%1

4.4 Statistics of mSQuAD

We were able to substantially expand the amount of data. These improvements to the translation pipeline not only enhanced the overall quality and consistency of the translated content but also increased the final volume of valid, usable examples. As a result, compared to the earlier version of the dataset presented in the study by [4], called gSQuAD. We now have 28% more data. A detailed breakdown of these updated data

statistics, including separate figures for both the development and training subsets in Table 4.5, clearly illustrates the gains achieved through this refined translation and error-correction process.

Table 4.5: *The translation output statistic of mSQuAD*

Statistic	Dev	Train
SQuAD Total q-a pair	34726	87599
Raw a-in-c after Translation	21530	54311
Edited q-a pair	5796	11323
Usable q-a pair	27326	65634
Increased Rate with Post-Translation Process	+%16	+%12
Increased Rate than gSQuAD	+%27	+%25
SQuAD Total Usable q-a pair Rate	%78	%75

To the remaining data, we discovered that several issues required manual intervention before the import could proceed. In particular, we identified a variety of distinct error types occurring across multiple scenarios. These errors ranged from formatting inconsistencies to missing values. A detailed overview of these problems, together with the specific causes and the corresponding need for human correction, is summarized in Table 4.6.

4.5 Evaluation Dataset with Deep Learning Model

4.5.1 BERT

In recent years, transformer-based models have demonstrated remarkable performance on a wide range of QAg tasks. This success is primarily attributed to their robust neural architectures and their ability to learn rich, high-dimensional language representations from massive text corpora. Among these models, BERT stands out as one of the most influential and widely adopted. When BERT was first introduced, it achieved state-of-the-art results on numerous NLP benchmarks, including tasks in natural language understanding, text classification, and QA [10].

Table 4.6: *Needed human operations after translation*

Answer-En	Answer-Tr	Content-Tr	Type
pressure terms	basınç koşulları	...basınç terim- lerinin...	in-consistency
second	saniye	...ikinci sezon...	synonymy
linebacker	Linebacker	...defans oyun- cusu...	not translated
the metric slug	metrik sümüklü- böğürtlen	metrik sümüklü- böcek	wrong translated
broken arm	kırık kol	...kırık bir kolla...	language differ- ences
grass	çimen	...çimin bir kısmı...	language differ- ences

BERT follows a semi-supervised learning paradigm. In the initial pre-training phase, the model is exposed to a large unlabeled corpus, such as word-sentence relationships. After this general language understanding is acquired, a fine-tuning phase is applied. BERT is further trained on specific downstream tasks using labeled data. In this step, task-specific layers are added on top of BERT, and the entire model is trained end-to-end for tasks such as QA, sentiment analysis, or Named Entity Recognition (NER).

Architecturally, BERT is based on the transformer architecture and primarily uses a stack of transformer encoder layers that rely on self-attention. These self-attention components allow the model to weigh each token's importance relative to all others, capturing long-range dependencies and subtle contextual cues more effectively than traditional recurrent or convolutional models. Overall, this architectural design enables BERT to achieve a more accurate and comprehensive understanding of time-consuming, computationally intensive applications.

4.5.2 Fine-Tuned Model

To train a BERT-based Turkish language model from scratch is highly time consuming and computationally intensive, so this study employs the pre-trained BERTURK model [41] as the Turkish large language model. BERTURK is a Turkish adaptation of the original BERT architecture and has been trained on an extensive, diverse collection of Turkish texts. Its training data consists of a cleaned, sentence-segmented version of the Turkish OSCAR corpus, a recent dump of the Turkish Wikipedia, several corpora from the OPUS collection, and an additional specialized corpus explicitly prepared for this task. Altogether, this training corpus amounts to approximately 35GB of textual data and includes 4,404,976,662 tokens, which provides a rich linguistic resource for modeling Turkish.

In this work, BERTURK serves as the backbone model for fine-tuning two QA systems. Using the mSQuAD-train-tr and gSQuAD-train-tr datasets whose detailed statistics are presented in Table 4.7. We fine-tune the BERTURK model to obtain the m-QA-Model-Tr and g-QA-Model-Tr configurations, respectively. The overall fine-tuning pipeline and architecture are illustrated in Figure 4.1. During this process, we adopt the BERTBASE configuration in the PyTorch framework, enabling efficient experimentation and integration with existing deep learning tools. The fine-tuning procedure, including all training epochs and optimization steps, required roughly 30 hours to complete.

4.5.3 Results and Evaluation

The m-QA-Model-Tr and g-QA-Model-Tr models were evaluated through a series of comprehensive test scenarios to better understand their behavior and performance. As an initial step, both models were run on the full mSQuAD-dev-tr dataset to obtain a general performance baseline. Following this, we conducted a more fine-grained evaluation by focusing on the subset of questions that are common to both the gSQuAD and

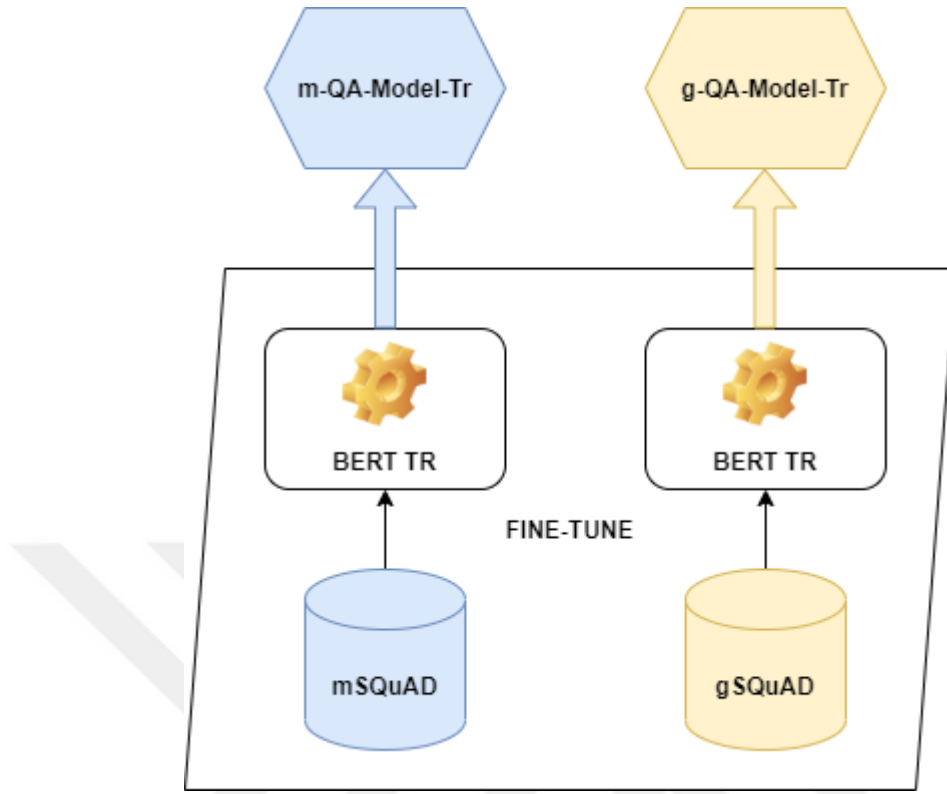


Figure 4.1: *Trained two QA models (m-QA-Model-Tr, g-QA-Model-Tr) based on mSQuAD and gSQuAD*

mSQuAD datasets. We first evaluated the models on mSQuAD-dev-tr. Then we repeated the evaluation on gSQuAD-dev-tr. This process allowed us to observe how each model performed across datasets with shared content but potentially different linguistic or contextual nuances.

The models were also examined from several detailed analytical perspectives using the common-question subset in mSQuAD-dev-tr. Specifically, we broke down performance by question type (e.g., “who” “when” “what” and other interrogative forms), which helped us identify whether certain question types. In addition, we analyzed Bilingual Evaluation Understudy (BLEU) scores to approximate translation quality and to see how closely the generated answers matched reference translations. We also considered the token length of the combined context-question-answer (c-q-a) sequences. Longer or

Table 4.7: *SQuAD, mSQuAD and gSQuAD Dataset Question/Title/Paragraph Count Statistics.*

Dataset	Question	Title	Paragraph
SQuAD-dev-en	10570	42	2067
mSQuAD-dev-tr	8197	42	2067
gSQuAD-dev-tr	5858	42	2067
SQuAD-train-en	87599	448	18896
mSQuAD-train-tr	65634	448	18896
gSQuAD-train-tr	40014	448	18896

more complex sequences may influence comprehension and answer extraction. Also, we grouped results by paragraph titles to investigate whether particular topics, domains, or document structures affected model performance.

Our overall analysis indicated that the model trained directly on the mSQuAD dataset consistently achieved better EM and F1 scores than its counterpart. The observed gains typically range from 1%–2%. Detailed comparative statistics, including per-metric and per-category breakdowns, are provided in Table 4.8.

In our setup, questions that the m-QA-Model-Tr failed to answer correctly in mSQuAD-dev-tr were replaced with their corresponding questions from the gSQuAD-dev-tr dataset. At the same time, the reverse substitution was also performed (i.e., questions that the g-QA-Model-Tr struggled with were replaced by their mSQuAD-dev-tr counterparts). This setup led to a notable increase in the proportion of correct answers.

Moreover, we observed a generally positive relationship between the quantity of training data and model performance as the volume of data was increased by combining examples from both mSQuAD and gSQuAD. The models tended to yield higher EM and F1 scores. This pattern underscores the importance of rich, diverse, and sufficiently large training corpora for QA systems.

We conducted a detailed analysis of how the lengths of the content (c), questions

Table 4.8: *m-QA-Model-Tr* and *g-QA-Model-Tr* models results

Model	EM	F1	Test Dataset
m-QA-Model-Tr	0.49	0.57	mSQuAD dev-tr
g-QA-Model-Tr	0.47	0.55	mSQuAD dev-tr
m-QA-Model-Tr	0.52	0.59	mSQuAD-dev-tr (only common questions)
g-QA-Model-Tr	0.50	0.58	mSQuAD-dev-tr (only common questions)
m-QA-Model-Tr	0.50	0.57	gSQuAD-dev-tr (only common questions)
g-QA-Model-Tr	0.48	0.56	gSQuAD-dev-tr (only common questions)
m-QA-Model-Tr	0.56	0.63	Question-Change at fails
g-QA-Model-Tr	0.55	0.62	Question-Change at fails

(q), and answers (a) influence the model’s behavior. We observed that extending any of these components generally reduces the model’s overall success. In other words, as the number of tokens in the question or the answer increases, the model becomes less likely to produce correct or satisfactory outputs. We computed accuracy scores based on the average token limits for each of the three input types. The resulting accuracy metrics, along with the associated token statistics, are systematically summarized and reported in Table 4.9, which provides a comprehensive overview of how varying input lengths impact both success and stability.

Table 4.9: *m-QA-Model-Tr* results on *mSQuAD-dev-tr* with tokenize count perspective as *c* =content, *q*= question, *a*= answer

Model	EM	F1	Token Count	Type
m-QA-Model-Tr	0.54	0.62	≤ 95	c
m-QA-Model-Tr	0.45	0.51	≥ 95	c
m-QA-Model-Tr	0.52	0.60	≤ 7	q
m-QA-Model-Tr	0.49	0.56	≥ 7	q
m-QA-Model-Tr	0.54	0.60	$= 1$	a
m-QA-Model-Tr	0.43	0.52	$= 2$	a
m-QA-Model-Tr	0.37	0.47	$= 3$	a

It is important to emphasize that each paragraph’s content is linked to a specific

domain. These domains can significantly influence model performance. In our analysis, we found that language models generally handle broad, everyday subject matter, such as information about countries, major cities, historical figures, and general biographies, more effectively with greater accuracy. These types of topics are typically well-represented in training data and use everyday vocabulary. They makes them easier for models to process. In contrast, we observed that paragraphs grounded in specialized or highly technical areas, especially those involving scientific terminology, niche disciplines, or domain-specific jargon, tend to pose greater difficulties. Models are more prone to misunderstand, oversimplify, or misrepresent content in these specialized domains. A detailed breakdown of these observations, including performance differences across domains in Tables 4.10 and 4.11.

Table 4.10: *m-QA-Model-Tr results on mSQuAD-dev-tr with domain oriented, top-5, bottom-5*

Domain	EM	F1	EM Rank
Islamism	0,72	0,77	1
1973 oil crisis	0.68	0.74	2
Kenya	0.66	0.76	3
Harvard University	0.63	0.63	4
Sky (United Kingdom)	0.62	0.66	5
Civil disobedience	0.38	0.47	44
European Union law	0.36	0.53	45
Prime number	0.35	0.43	46
Chloroplast	0.34	0.48	47
Computational theory	0.34	0.51	48

The questions were another point of observation for us. In particular, the models tended to perform well on simple, well-defined questions with unambiguous answers, such as those beginning with "when", "how much", and "who". For these types of questions, the models could reliably identify specific information and return correct responses. However, their performance declined noticeably when they were presented with

Table 4.11: *g-QA-Model-Tr results on mSQuAD-dev-tr with domain oriented, top-5, bottom-5*

Domain	EM	F1	EM Rank
United Methodist Church	0,72	0,74	1
1973 oil crisis	0.68	0.76	2
Islamism	0.67	0.76	3
Kenya	0.64	0.76	4
Nikola Tesla	0.63	0.70	5
Immune system	0.36	0.47	44
Ctenophora	0.35	0.47	45
Oxygen	0.34	0.46	46
Packet switching	0.32	0.41	47
Computational theory	0.30	0.43	48

more open-ended or less clear questions, such as those starting with ”which” and ”what”. These question types often require a broader understanding of context or the consideration of multiple possible correct answers. All of them posed difficulties for the models. A comprehensive breakdown of the models’ performance across these question categories in Tables 4.12 and 4.13.

Table 4.12: *m-QA-Model-Tr results on mSQuAD-dev-tr with question type, q type count ≥ 100*

Question Type	EM	F1	EM Rank	Count
Ne zaman (When)	0,76	0,80	1	423
Kaç (How much)	0.69	0.73	2	443
Ne kadar (How much)	0.62	0.67	3	117
Kim	0.57	0.63	4	570
Nerede (Where)	0.45	0.52	5	151
Hangi (Which)	0.51	0.59	6	996
Ne (What)	0.42	0.52	7	1661

To assess the overall quality of the translations produced in our experiments, we relied on the consistently used gQuAD-dev-tr dataset as a primary reference. Then we

Table 4.13: *g-QA-Model-Tr* results on *mSQuAD-dev-tr* with question type, *q* type count ≥ 100

Question Type	EM	F1	EM Rank	Count
Ne zaman (When)	0,72	0,75	1	423
Kaç (How much)	0.69	0.72	2	443
Ne kadar (How much)	0.57	0.64	3	117
Kim	0.56	0.62	4	570
Nerede (Where)	0.53	0.62	5	151
Hangi (Which)	0.52	0.59	6	996
Ne (What)	0.40	0.50	7	1661

extended our evaluation to the additional *mSQuAD-dev-tr* development set. For this second set, we computed the BLEU score as a quantitative measure of translation accuracy. We observed that lower BLEU scores are associated with decreased model accuracy. A detailed summary of these results, BLEU scores in Table 4.14.

Table 4.14: *m-QA-Model-Tr* results with BLEU as *c* =content, *q*= question, *a*= answer

Model	EM	F1	BLEU	Type
m-QA-Model-Tr	0.54	0.61	≥ 0.75	c
m-QA-Model-Tr	0.45	0.51	≤ 0.75	c
m-QA-Model-Tr	0.54	0.61	≥ 0.75	q
m-QA-Model-Tr	0.47	0.54	≤ 0.75	q
m-QA-Model-Tr	0.53	0.60	$= 1$	a
m-QA-Model-Tr	0.40	0.51	≤ 1	a

5. CREATING BASELINE MODELS AND ANALYSIS OF RESULTS

5.1 Bidirectional Encoder Representations Transformer Baseline

One of our baseline models is the output of the BERT architecture, as explained in detail in the previous section. In recent years, transformer-based models have been used in question-answering tasks owing to their robust architecture. Upon its introduction, BERT achieved state-of-the-art accuracy in various NLP tasks, including language understanding and QA. BERT uses a semi-supervised learning approach.

As training a large language model is costly and not the focus of our study, the current BERTurk¹ large language model was used. The current BERTurk version was trained on a filtered, sentence-segmented Turkish OSCAR corpus, a recent Wikipedia dump, various OPUS corpora, and a special corpus. Its corpus size is 35GB with 4,404,976,662 tokens. In the context of the BERT architecture for QA tasks, we established a baseline for fine-tuning using our mSQuAD dataset. It was called the BERT baseline. The process involves feeding the mSQuAD dataset with the BERTurk model for fine-tuning. The output of this fine-tuning step is a new model called BERT-Baseline-Model-TR. You can see the basic flow showing the baseline in Figure 5.1.

5.2 OpenAI ChatGPT & Qwen2 Baselines

OpenAI is a prominent Artificial Intelligence (AI) research and deployment organization founded in December 2015 and headquartered in San Francisco, California. The organization has positioned itself at the forefront of AI innovation. They try to achieve Artificial General Intelligence (AGI). In this context, AGI is defined as "highly autonomous systems that outperform humans at most valuable work".

¹BERTurk, <https://github.com/stefan-it/turkish-bert>

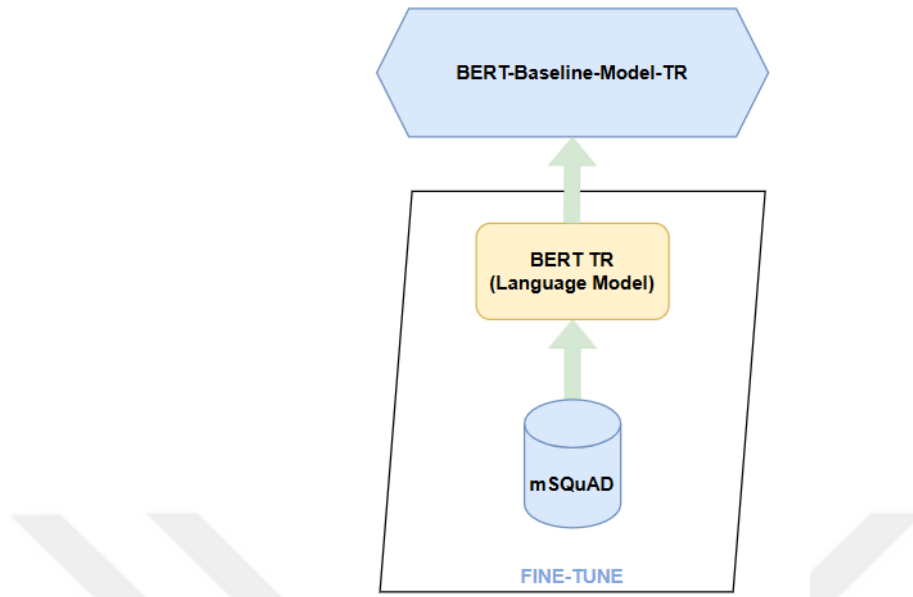


Figure 5.1: *BERT Baseline, fine-tune operation with BERTurk on mSQuAD Dataset*

OpenAI has developed the GPT series of large language models, which have significantly shaped the current landscape of Generative AI. ChatGPT was launched publicly in November 2022. The debut of ChatGPT sparked intense public and academic interest in Generative AI. Within the system described here, two versions of ChatGPT 3.5 Turbo and ChatGPT 4.1 have been integrated as baseline models. These baselines serve as strong reference points, enabling systematic comparison of system performance.

The system also incorporates the open-source Qwen2 (Qwen/Qwen2-7B-Instruct)² language model as an additional baseline. Qwen2 represents the newest generation within the Qwen family of large language models and reflects ongoing advancements in open-source AI research. The repository associated with this work specifically hosts a variant of Qwen2 with 7 billion parameters.

Qwen2’s abilities across a broad spectrum of tasks, including open-ended text generation, multilingual processing, computer programming, mathematical problem-solving,

²Qwen2, <https://huggingface.co/Qwen/Qwen2-7B-Instruct>

and various logical reasoning challenges. Qwen2 has repeatedly demonstrated strong performance, surpassing the majority of existing open-source models.

All these models operate within the overall Generative AI (GAI) framework alongside large language models, working together to perform complex tasks. GAI is a specialized branch of AI that relies on generative models to create new content such as text, images, videos, or other types of data. These models are trained on large datasets and learn the underlying structures, patterns, and relationships within those datasets. Afterwards, they use this learned knowledge to generate new, coherent data based on the input they receive, which is most often provided in the form of natural-language prompts.

Prompt engineering is the process of designing and structuring these inputs so that they can be clearly understood and effectively processed by a Generative AI model. A prompt is a piece of natural-language text that specifies the task the AI should perform. For example, it could be a direct query such as "What did Newton discover?", an instruction or request like "Can you write a poem that describes love like Shakespeare?". The form and clarity of the prompt significantly influence the quality and accuracy of the model's response.

The ChatGPT generative models we employ via the OpenAI infrastructure generally require carefully crafted, relatively strong prompts supplied externally to produce complete and accurate answers. This need for precise prompting is equally valid for the Qwen2-based system. The expected output is not a free-form response in SQuAD tasks. A specific answer may be a word or a short sequence of words that is found explicitly within a given paragraph. When using generative models like ChatGPT and Qwen2 for such tasks, two challenges occurred.

The first challenge was to ensure that the model's response is grounded in the provided paragraph. Because generative AI models are trained on extensive corpora, they tend to generate answers based on patterns in their training data. To make the system

respond strictly by referencing the given paragraph, it must be carefully steered through explicit prompt instructions so that it focuses on the supplied context and does not hallucinate information.

The second challenge is that the system must return the exact answer span and produce a semantically similar paraphrase. In SQuAD task, even a very close approximation is considered incorrect if it does not match the expected text span. This requirement also needs to be encoded in the prompt so the model knows to extract and repeat the exact wording rather than rephrase or elaborate.

Both of these issues can be mitigated through well-designed prompt guidance. To handle them, ChatGPT and Qwen2 are each given a combination of inputs: a question, a supporting paragraph, and two distinct prompts. The first prompt constrains the model to base its answer solely on the information contained in the paragraph. It disallows reliance on outside knowledge. The second prompt instructs the model to provide the answer using the exact words or phrase from the paragraph, instead of returning a similar or reworded sentence. The specific forms of these prompts are presented in Table 5.1.

In the baseline system architecture, the content passages, questions, and corresponding prompts are fed into two versions of the ChatGPT model (ChatGPT 3.5 Turbo and ChatGPT 4.1) and the Qwen2 model. Each model receives the same set of inputs so that their outputs can be fairly compared. The questions, when paired with the paragraphs and the guiding prompts, are passed to the models, which then generate responses intended to be both context-bound and textually precise. In this way, the models are driven to produce meaningful, accurate outputs that align with the strict requirements of SQuAD-style question-answering tasks.

Table 5.1: *Baselines prompt samples*

	Applied Prompts
Prompt-1	You must answer the following question, taking into account the data in the paragraph.
Prompt-2	The answer should be given in words only, not in sentences.

5.3 Baseline Results

The first baseline model was developed using the BERTurk architecture within a deep learning framework. This model was then fine-tuned on the SQuAD (using the mSQuAD dataset) dataset to adapt it specifically for QA in the target language and domain. Following the construction of this BERT-based baseline, three additional baseline models were introduced to enable a broader comparison: two based on the ChatGPT family of models and one based on the open-source Qwen2 model. Among the ChatGPT-based systems, one uses the ChatGPT 3.5 Turbo model, while the other relies on ChatGPT 4.1 (release version: gpt-4.1-2025-04-14), which is considered one of the most advanced, accurate, and up-to-date large language models currently available for such tasks. The Qwen2 model was selected as a representative open-source language model. It is widely recognized for its strong performance and accessibility to the research community.

Their outputs were compared using two commonly accepted metrics in question-answering research: EM and F1 score for these four baseline models BERT, ChatGPT 3.5 Turbo, ChatGPT 4.1, and Qwen2. The EM metric measures the percentage of predictions that match the ground-truth. It provides a strict indicator of correctness. The F1 score is the harmonic mean of precision and recall and thus captures partial overlap between predicted and true answers. The detailed EM and F1 values obtained for each of the four baseline models are presented in Table 5.2. They serve as the primary reference for the quantitative performance comparison.

When examining these results, it becomes apparent that the BERT baseline model outperforms the ChatGPT 3.5 Turbo baseline by approximately 4% in EM. However, the difference between the two models in terms of F1 score is relatively small, indicating that although ChatGPT 3.5 Turbo may not always produce perfectly matching answers. It often generates responses that are close to the correct ones. As anticipated, ChatGPT 4.1 achieved the best performance among all evaluated systems, the highest EM and F1 scores. This confirms the expectation that newer and larger models tend to yield more accurate and robust results on complex question-answering tasks. Although the Qwen2 model is a strong open-source alternative, it achieved lower EM and F1 scores than all other baselines.

Table 5.2: *Results of baseline models on mSQuAD*

Model	EM	F1
Bert Baseline Model (B1)	0.49	0.57
ChatGPT 3.5 Baseline Model (B2)	0.45	0.56
ChatGPT 4.1 Baseline Model (B3)	0.57	0.67
Qwen2-7B-Instruct Baseline Model (B4)	0.34	0.44

Baseline results were examined in detail from multiple perspectives to gain a comprehensive understanding of how different factors influence model behavior. In this process, we identified several factors that affect model performance, among which the length of the input data proved particularly critical. The size of the content provided to the system, whether in the form of a paragraph, a question, or an answer, plays a key role in shaping the model’s effectiveness. Ideally, a successful model should exhibit stable, consistent performance regardless of variations in input length, maintaining reliability across both short and long pieces of text.

We observed that extended input lengths tend to have a detrimental effect on overall model success. As paragraphs, questions, or answers grew longer, the models showed

a noticeable decrease in accuracy and robustness. To explore this phenomenon in more depth, we examined the separate contributions of content length (c), question length (q), and answer length (a) to model performance.

To quantify these observations, we calculated accuracy values based on average token limits for each of the three input types: content, questions, and answers. These computed accuracy scores, organized by token range, are reported in Table 5.3. It provides an empirical basis for understanding how input length systematically influences model performance.

Table 5.3: *BERT, ChatGPT, Qwen2 baselines with token-count (c = content, q = question, a = answer), Baseline = B, Token Size = #, Token Type = T*

B1	EM	F1	B2	EM	F1	B3	EM	F1	B4	EM	F1	#	T
BERT	0.54	0.62	GPT3.5	0.44	0.56	GPT4.1	0.57	0.67	Qwen2	0.33	0.44	< 95	c
BERT	0.45	0.51	GPT3.5	0.45	0.56	GPT4.1	0.58	0.68	Qwen2	0.34	0.44	≥ 95	c
BERT	0.52	0.60	GPT3.5	0.54	0.57	GPT4.1	0.58	0.66	Qwen2	0.32	0.43	< 7	q
BERT	0.49	0.56	GPT3.5	0.44	0.56	GPT4.1	0.57	0.68	Qwen2	0.34	0.44	≥ 7	q
BERT	0.54	0.60	GPT3.5	0.47	0.58	GPT4.1	0.63	0.73	Qwen2	0.34	0.44	= 1	a
BERT	0.43	0.52	GPT3.5	0.49	0.60	GPT4.1	0.64	0.75	Qwen2	0.33	0.43	= 2	a
BERT	0.37	0.47	GPT3.5	0.42	0.57	GPT4.1	0.55	0.65	Qwen2	0.34	0.45	= 3	a

Language models tend to perform more reliably on general-purpose material such as descriptions of locations, information about cities, and biographical narratives because these topics are widely represented in their training data. In contrast, they encounter notable difficulties when dealing with material that uses dense or specialized terminology.

This issue is particularly evident when the models are run on articles that focus heavily on technical jargon, highly abstract ideas, or domain-specific language, including scientific terminology, discussions in the arts, and complex philosophical arguments. A detailed breakdown of these observations and performance differences is presented in Table 5.4.

When constructing Table 5.4, we selected a subset of domains in which BERT

performed exceptionally well, as well as several domains where its performance was noticeably weak. The “Baseline1 Rank” column captures BERT’s relative ranking across these domains. They are highlighting the five best-performing and five worst-performing domains. Using these rankings as a reference, we evaluated and compared the performance of the ChatGPT and Qwen2 baseline models on a domain-by-domain basis.

Table 5.4: Domain based evaluation - BERT, ChatGPT, Qwen2 Baselines, Baseline= B

Domain	B1	EM	F1	B2	EM	F1	B3	EM	F1	B4	EM	F1	B1 Rank
Islamism	BERT	0,72	0,77	GPT3.5	0,38	0,51	GPT4.1	0,51	0,62	Qwen2	0,31	0,41	1
1973 oil crisis	BERT	0,68	0,74	GPT3.5	0,48	0,60	GPT4.1	0,61	0,66	Qwen2	0,45	0,53	2
Kenya	BERT	0,66	0,76	GPT3.5	0,43	0,56	GPT4.1	0,44	0,54	Qwen2	0,28	0,43	3
Harvard University	BERT	0,63	0,63	GPT3.5	0,61	0,71	GPT4.1	0,66	0,73	Qwen2	0,51	0,57	4
Sky (United Kingdom)	BERT	0,62	0,66	GPT3.5	0,50	0,62	GPT4.1	0,65	0,74	Qwen2	0,36	0,43	5
Civil disobedience	BERT	0,38	0,47	GPT3.5	0,44	0,50	GPT4.1	0,46	0,57	Qwen2	0,34	0,40	44
European Union law	BERT	0,36	0,53	GPT3.5	0,36	0,52	GPT4.1	0,52	0,62	Qwen2	0,26	0,42	45
Prime number	BERT	0,35	0,43	GPT3.5	0,28	0,40	GPT4.1	0,47	0,59	Qwen2	0,23	0,38	46
Chlorop- last	BERT	0,34	0,48	GPT3.5	0,26	0,38	GPT4.1	0,38	0,51	Qwen2	0,18	0,32	47
Computa- tional theory	BERT	0,34	0,51	GPT3.5	0,26	0,38	GPT4.1	0,42	0,54	Qwen2	0,14	0,25	48

Model performances showed notable differences depending on the specific type of question they were asked to answer. The models handled relatively straightforward, fact-based questions quite well. In contrast, they had considerably more difficulty with more open-ended question types. A comprehensive breakdown of performance across these question categories is presented in Table 5.5.

Table 5.5: BERT, ChatGPT, Qwen2 baselines with question type, samples from question type count ≥ 100 , Baseline= B

Question Type	B1	EM	F1	B2	EM	F1	B3	EM	F1	B4	EM	F1	B1 Rank
Ne zaman (When)	BERT	0,76	0,80	GPT3.5	0,72	0,60	GPT4.1	0,77	0,85	Qwen2	0,27	0,43	1
Kaç (How much)	BERT	0,69	0,73	GPT3.5	0,48	0,33	GPT4.1	0,47	0,63	Qwen2	0,24	0,33	2
Ne kadar (How much)	BERT	0,62	0,67	GPT3.5	0,49	0,34	GPT4.1	0,46	0,63	Qwen2	0,19	0,31	3
Kim (Who)	BERT	0,57	0,63	GPT3.5	0,72	0,62	GPT4.1	0,73	0,79	Qwen2	0,57	0,65	4
Nerede (Where)	BERT	0,45	0,52	GPT3.5	0,55	0,43	GPT4.1	0,51	0,63	Qwen2	0,19	0,34	5
Hangi (Which)	BERT	0,51	0,59	GPT3.5	0,65	0,54	GPT4.1	0,65	0,73	Qwen2	0,46	0,56	6
Ne (What)	BERT	0,42	0,52	GPT3.5	0,48	0,36	GPT4.1	0,50	0,61	Qwen2	0,28	0,39	7

In analyses of the baseline models, it was observed that their domain-based accuracy values fluctuated noticeably. More specifically, the accuracy scores varied not only

across different domains but also according to several structural properties of the input. For instance, the models' performance differed depending on the length of the input data and the formulation of the questions. When the questions were categorized into types, it became evident that each type yielded distinct accuracy levels. Consequently, it is critically important to design and maintain models with a balanced, robust structure that can adapt effectively to changing data content. Such a balance helps ensure greater consistency in accuracy, reduces performance drops under specific conditions, and supports more reliable communication between domains.



6. EXPERIMENTS WITH STRUCTURED AND ENRICHED DATA

6.1 Creating Structured Paragraph Data

When we examined the baseline results in detail, we observed that the models' performance fluctuated significantly across specific paragraphs or questions. This variability made it challenging to identify the exact source of the problems. In practice, large language models (e.g. BERTurk, Qwen2), transformer-based models (such as BERT), and SaaS services (such as those provided by OpenAI and ChatGPT) all operate as complex, large-scale black-box systems. Their decision processes are not easily interpretable. We noticed the data's effect on model behavior and output quality. This consistent pattern led us to prioritize efforts to structure the data more systematically and explicitly, and stabilize performance.

In natural language understanding, humans typically interpret language by mentally decomposing sentences into individual words or phrases, then inferring relationships among these components to derive the sentence's overall meaning. To represent this process in a form suitable for computational systems, the elements of a sentence can be modeled as a graph. You can think of a graph as words being nodes and relationships being links (edges). When information in a domain is stored and organized using this graph structure, the result is called a knowledge graph. A knowledge graph consists of nodes and links, where nodes represent entities or concepts and links capture their relationships. These relationships are often stored in a triple format: subject, predicate, and object. They provide a clear, machine-readable representation of semantic relations. In this representation, the subject and object components of the triple are stored as nodes. At the same time, the predicate, which expresses the type of relationship between them, is represented as a link connecting those nodes. This structured representation enables systematic encoding, querying, and reasoning about domain knowledge.

There are multiple approaches and methodologies available for constructing a knowledge base, each with its own strengths, weaknesses, and areas of applicability [42]. The selection of a suitable method depends on several key performance indicators (KPIs) and practical constraints. Including the nature and complexity of the problem being addressed, the desired coverage of the knowledge domain, the availability and quality of resources, the time available for the construction process, and the level of accuracy or precision required. Some of the commonly used knowledge base creation methods, along with their main KPIs, properties, and influential characteristics, are summarized in Table 6.1. These methods may include manual curation by domain experts, semi-automatic construction using rule-based systems, or traditional NLP pipelines. After evaluating these methods against the aims of our study and the constraints mentioned above, we concluded that the most practical and effective strategy was to adopt an LLM-assisted knowledge-creation approach. This approach leveraged the generative and interpretive capabilities of large language models to support the extraction, structuring, and enrichment of domain knowledge while maintaining a level of control and validation aligned with our research objectives.

- LLM-Assisted: Great for quickly filling a KB from external sources, but needs close monitoring to prevent hallucinations.
- Manual Creation: Very accurate and reliable, but hard to scale and costly in maintenance and human labor.
- (Information Retrieval) IR-Based Methods: Extremely scalable and efficient for coverage-focused workloads.
- Traditional NLP Pipelines: Particularly powerful for structured extraction in specialized domains.

Table 6.1: *Evaluation of knowledge base creation technique, ranking(1 to 10, Low:0-3, Medium:4-7, High:8-10)*

Technique	Accuracy	Coverage	Human Effort	System Cost
LLM-Assisted	High (9)	High(9)	Low(3)	Medium(6)
Manual Creation	High (10)	Low(3)	High (10)	Low(2)
IR-Based Methods	Medium(7)	Medium(7)	Medium(5)	Medium(5)
Traditional NLP Pipelines	High (8)	Medium(7)	Medium(5)	Medium(5)

ChatGPT, OpenAI, and the LangChain ecosystem have recently introduced an integrated pipeline that significantly accelerates the process of overcoming this challenge. This pipeline provides a robust, scalable infrastructure for converting raw, unstructured text into well-defined, structured data that conforms to a predefined schema. We generated rich knowledge graphs from unstructured paragraph-level data. The ChatGPT language model plays a central role in this process: it can identify and extract key entities embedded in the text and infer the semantic relationships linking them. The LangChain libraries take these model outputs and systematically transform them into structured graph representations that conform to a knowledge graph model.

The complete workflow for generating structured data from unstructured text is illustrated in detail in Figure 6.1. First, paragraph data are ingested into the system as input, with the explicit goal of converting them into a structured representation. Alongside this textual input, we also provide our graph schema specification, such as node types, relationship types, and relevant attributes, by encoding it directly into prompts. This prompt-engineering step guides the language model to produce outputs aligned with the desired graph structure.

To operationalize this pipeline, we rely on both the LangChain¹ framework and

¹LangChain, <https://www.langchain.com/>

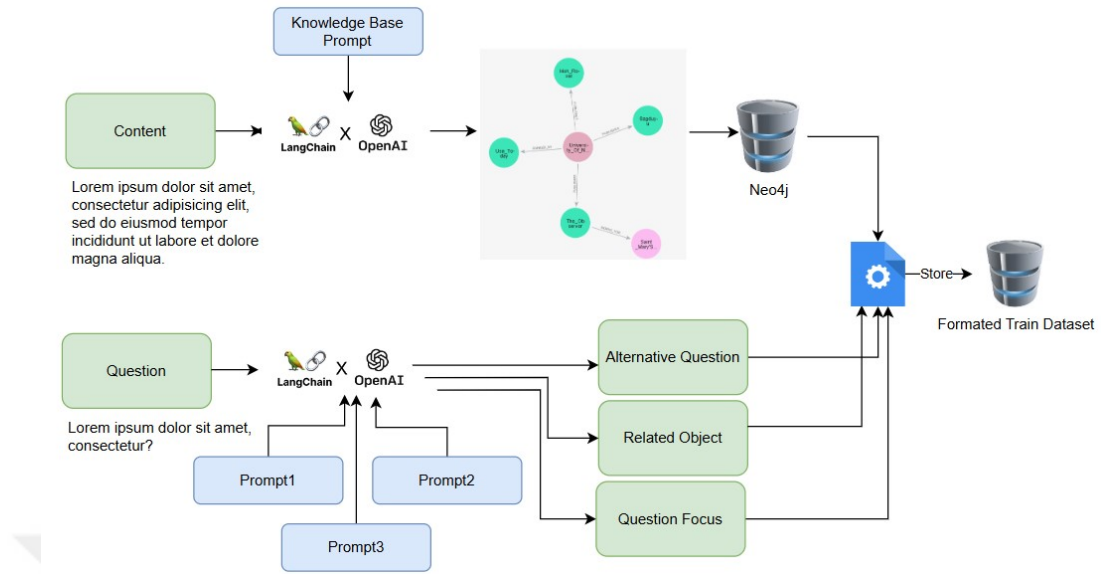


Figure 6.1: *Process of structuring data transformation*

the ChatGPT infrastructure to process inputs and derive structured outputs from unstructured data. Langchain is an orchestration layer for designing and executing controllable, agentic workflows, enabling us to chain multiple model calls. Through LangChain, we can manage the flow from raw text ingestion to an entity-relationship extraction graph.

When the structured representations have been generated, we store them in a dedicated graph database, specifically the Neo4j database². Neo4j is a graph database management system (GDBMS) developed by Neo4j. It is designed to efficiently store, query, and traverse highly connected data. By loading the processed and structured outputs into Neo4j, we can run complex graph queries, execute graph algorithms, and visualize entity-relationship networks. This overall pipeline from unstructured paragraphs provides a comprehensive, extensible solution for converting free-form text into actionable, structured knowledge.

The process by which raw paragraph data are converted into a structured representation is illustrated in Table 6.2. When a paragraph is first input into the system, it

²Neo4J, <https://neo4j.com/>

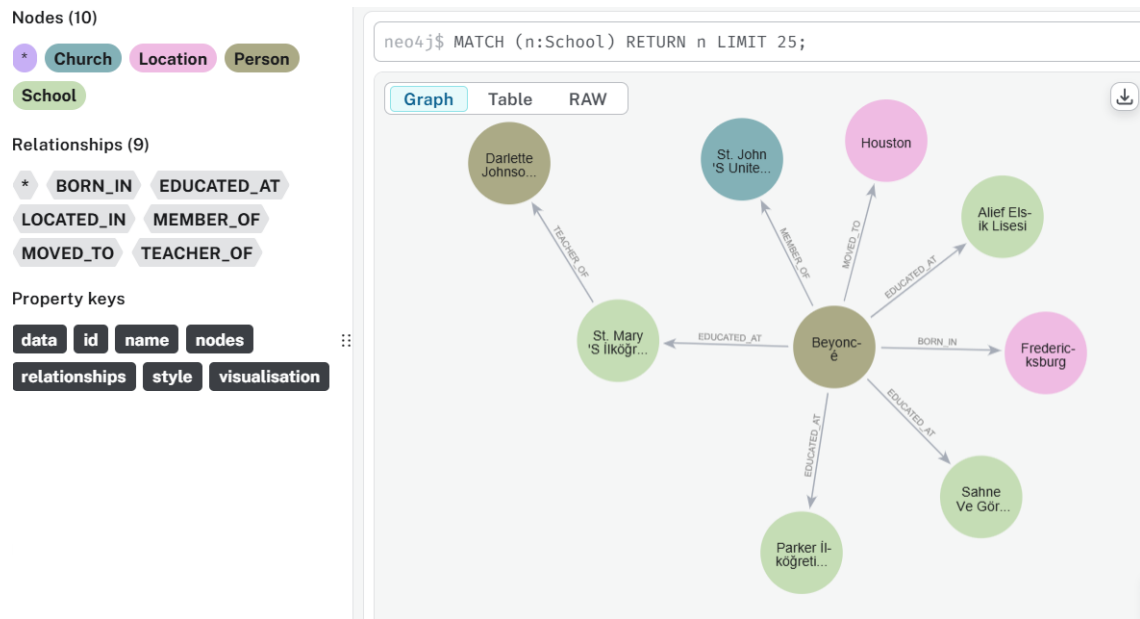


Figure 6.2: Sample of knowledge graph data

undergoes a series of processing steps involving tools and frameworks such as ChatGPT and LangChain. Through these steps, the original unstructured text is transformed into a structured schema that corresponds to the format depicted in the "Output" column of Table 6.2 and visually summarized in Figure 6.1. The resulting representation takes the form of graph data composed of numerous triples, each capturing a specific relationship between entities, and these triples are subsequently stored in a Neo4J database for efficient querying and management. In the later training and testing stages, this stored graph data referred to as "Knowledge Base Graph Data" is reconverted into paragraph-style text, which appears in Table 6.2 as the content of the "Output" column for the corresponding row. In this way, the system supports a bidirectional flow between unstructured text and structured graph-based knowledge.

Table 6.2: *Transformation of structured data sample*

Data	Output
Beyoncé, Fredericksburg, Teksas'taki St. Mary's İlköğretim Okulu'na gitti ve burada dans derslerine kaydoldu. Şarkı söyleme yeteneği, dans öğretmeni Darlette Johnson'ın bir şarkıyı mırıldanmaya başlamasıyla keşfedildi ve onu bitirdi, tiz notalara ulaşabildi. Beyoncé'nin müziğe ve performansa olan ilgisi, yedi yaşında John Lennon'ın "Imagine" şarkısını söyleyerek 15/16 yaşındakileri yenmek için bir okul yetenek gösterisini kazandıktan sonra da devam etti. 1990 sonbaharında Beyoncé, Houston'daki bir müzik mıknaatısı okulu olan Parker İlköğretim Okulu'na kaydoldu ve burada okulun korosuyla birlikte sahne alacaktı. Ayrıca Sahne ve Görsel Sanatlar Lisesi'ne ve daha sonra Alief Elsik Lisesi'ne gitti. Beyoncé ayrıca St. John's United Methodist Kilisesi'nde iki yıl boyunca solist olarak koronun bir üyesiydi.	Figure 6.2 shows output of knowledge graph
Knowledge Base Graph Data	Beyoncé, St. Mary's İlköğretim Okulu'na gitti. Beyoncé, Darlette Johnson'ı keşfetti. Beyoncé, John Lennon şarkısını seslendirdi. Beyoncé, Parker İlköğretim Okulu'na gitti. Beyoncé, Sahne Ve Görsel Sanatlar Lisesi'ne gitti. Beyoncé, Alief Elsik Lisesi'ne gitti. Beyoncé üyesi St. John's United Methodist Kilisesi.

6.2 Creating Enriched Question Data

After the paragraph was structured within the processing pipeline, we implemented a more comprehensive and enriched question–data procedure to improve the quality of the input questions. To enrich this question’s data, we first conducted a systematic evaluation of the effectiveness and overall success of several alternative enrichment methods. The influence of enrichment strategies on accuracy was assessed using the Baseline1 model as our reference point.

We considered three main options. Option-1 enriched the question text by issuing a prompt call to ChatGPT, allowing the model to reformulate and elaborate on the original questions. Option-2 relied on paraphrasing the questions via the T5 model³. It produces alternative phrasings while preserving semantic content. Option-3 focused on generating semantically similar variants by using the Turkish FastText model⁴ (cc.tr.300.vec) through the Gensim library, thereby leveraging word embeddings to locate and construct related question formulations.

We then applied these techniques to the Baseline1 model to phrase the questions in various ways and measured the resulting changes in performance. The results were analyzed by comparing the effects of using the original, unmodified questions in Baseline1 with those obtained when the same questions were altered using each enrichment technique. These comparative findings are presented in Table 6.3. Based on this analysis, we concluded that the most effective and reliable method was prompt–based ChatGPT enrichment. It yielded the best overall accuracy among the three options.

We should better understand how the specific wording of prompts influences both the model’s outputs and its accuracy. We designed four alternative prompt formulations. Although these prompts differed in wording, they were carefully crafted so that their

³T5 Multilingual Model mT5 multilingual XLSum, <https://huggingface.co/csebuetnlp/>

⁴Meta Fasttext, <https://ai.meta.com/tools/fasttext/>

intended meaning remained the same. Each of these four prompts was tested on the Base-line1 model to investigate whether minor wording changes would produce noticeable differences in performance. The resulting accuracy values, summarized in Table 6.4. Results showed only a negligible variation of approximately $\pm 1\%$ after rounding. Semantically equivalent prompts had minimal impact on the final accuracy.

We conducted several studies to identify which parts of the question are most crucial from different analytical perspectives. The insights from these investigations were organized into three main areas. The first item focuses on determining how a question can be reformulated or asked in alternative ways using prompts. The second item aims to identify the primary object or entity the question focuses on. The third item examines what the target object is related to within the context of the question, capturing the relationships, properties, or attributes that link the object to other elements in the sentence. By systematically collecting and evaluating data according to these three items, we gained a more detailed understanding of how question structure and emphasis affect the data. Model performance and enrichment techniques can be tailored to highlight the most informative components of each question.

- Alternative Question: Describes how we might ask this question differently.
- Answer Related Object: Which objects in the question could possibly be related to the answer.
- Important Object: Which is the most important object in the question of finding an answer.

We focused on resolving these challenges by leveraging ChatGPT alongside the LangChain framework, enabling us to structure and process queries systematically. For each question, we formulated and applied three distinct prompts, yielding three separate

outputs. The continuation of the study relied on the set of prompts presented in Table 6.5, which served as the standardized basis for further experimentation and analysis.

Alongside storing the original question in the database, we also saved three additional, enriched versions of the question. In this way, every question in the system is associated with multiple enriched representations, designed to capture various nuances and contextual expansions of the original query. Examples of these enriched question datasets for a range of questions are provided in Table 6.6, illustrating how the prompts produced more detailed responses.

Table 6.3: *Results of question enrichment trial methods*

Approach	EM	F1	Result
Bert Baseline Model	0.49	0.57	-
Prompt Calling (ChatGPT)	0.51	0.57	Partial better performance
Word2Vec Similarity (fast-Text)	0.41	0.50	Less accuracy
Paraphrasing (mT5)	0.11	0.12	Unsuccessful

Table 6.4: *Accuracy of different prompts on Baseline 1 for A alternative question*

Applied Prompt	F1	EM
Indicates that it describes how we might ask this question differently.	0.49	0.57
Suggests a way to rephrase the question to convey the same meaning.	0.49	0.57
Provides a different wording or structure for asking the same question.	0.48	0.56
Offers an alternative formulation of the original question while preserving intent.	0.50	0.56

Table 6.5: *Baseline ChatGPT prompt structure*

Type	Applied Structure Data&Prompts
Alternative Question	Indicates that it describes how we might ask this question differently.
Answer Related Object	Indicates that which objects in the question could possibly be related to the answer?
Important Object	Indicates that which is the most important object in the question to find the answer?

Table 6.6: *Sample of enriched question data*

Question	Alternative Question	Answer Related Object	Important Object
Notre Dame Scholastic Dergisi ne zaman yayınlanmaya başladı?	Notre Dame Scholastic Dergisi'nin ilk yayın tarihi ne zaman?	yayınlanma zamanı	Notre Dame Scholastic Dergisi
Notre Dame Ana Binası'nın önünde ne var?	Notre Dame Ana Binası'nın ön cephesinde yer alan yapı nedir?	önünde	Notre Dame Ana Binası
Notre Dame'daki Mühendislik Koleji hangi yılda kuruldu?	Notre Dame Üniversitesi Mühendislik Fakültesi'nin kuruluş yılı nedir?	kuruluş yılı	Mühendislik Koleji
Mendoza College of Business'daki lisans okulu BusinessWeek'e göre nerede sıralandı?	Mendoza College of Business'ın BusinessWeek sıralamasındaki yeri nedir?	lisans okulu	Business Week

6.3 Training with Structure Data

A novel model was created by carefully and iteratively refining the original BERT architecture, using the mSQuAD dataset as the primary benchmark for development and evaluation. In this enhanced architecture, two main categories of input were supplied: “Formatted KnowledgeBase Content Data” and “Enriched Question Data”. This approach represented a departure from conventional baseline training procedures.

In the proposed architecture, knowledge-based paragraph data was incorporated alongside the original training data, enabling the model to access more organized and semantically rich information during learning. At the same time, the original questions from the dataset were supplemented with additional, enriched forms of the same questions. This enriched question data consisted of several derived representations, including a distilled version that focused on the most relevant segment. All of these variants were fed into the model during training, allowing it to better understand nuanced question intent and to align different surface forms of queries with the same underlying meaning.

By combining structured knowledge passages with enriched question versions, the training aimed to improve the model’s understanding. A more thorough and systematic explanation of each architectural component, the nature of the input data, and the end-to-end processing pipeline will be provided in the following sections.

- BertTR: The pre-trained Turkish large language model.
- Training Data:
 - mSQuAD: Translated SQuAD dataset into Turkish.
 - Formatted KnowledgeBase Content Data: Structured paragraph data from coming knowledge base database.
 - Enriched Question Data: alternative question + answer related object + important object

- Fine-Tuning Process: BERT infrastructure
- Output (New BERT-Model-TR): New KnowledgeBase Model

As mentioned earlier, BERT is a transformer-based architecture with only an encoder. It offers a highly successful architecture, with multiple attention mechanisms and parallel input processing. The BERT-uncased model was used for training. Case sensitivity is not critical in question-answer systems. Furthermore, different features are learned in each of the 12 attention layers during training. The hidden capacity is 768, and the learning infrastructure has approximately 110 million parameters. From a data perspective, the original question and the enriched question data are fed into the model together. Similarly, the original content is fed into the model simultaneously with the structured content data. The general appearance of the architecture is illustrated in the Figure 6.3.

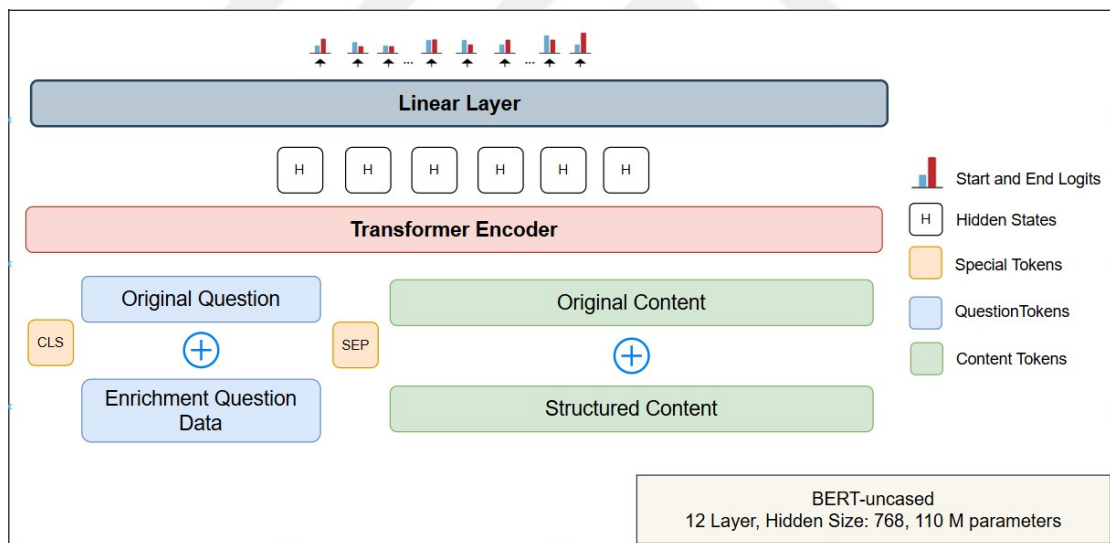


Figure 6.3: BERT Training with Structured Data

The input sequence is constructed by concatenating multiple components:

- A special classification token [CLS] placed at the beginning.
- The **Original Question** tokens.

- An additional **Enrichment Question Data** segment, which may include structured or augmented information related to the question.
- A separator token [SEP].
- The **Original Content** tokens.
- Additional **Structured Content**.

Each segment is embedded using token, position, and segment embeddings, allowing BERT to distinguish between question tokens and content tokens.

Prediction Layer A linear layer is applied on top of the hidden states to compute task-specific logits. For extractive question answering, this layer outputs:

- **Start logits** indicating the probability of each token being the start of the answer span,
- **End logits** indicating the probability of each token being the end of the answer span.

The effect of data enrichment and structuring on accuracy is examined in detail in the Results section.

7. RESULTS OBTAINED WITH STRUCTURED AND ENRICHED DATASETS

In our assessment of the baseline models, we noticed that their performance is not uniform; instead, the success rate of each model changes depending on several factors, such as the length of the input content, the specific domain to which the question belongs, and the type or structure of the question being asked.

This section focuses on a detailed evaluation of the outputs produced by our newly developed structure-based trained model, referred to as the KnowledgeBase model. We analyze how this model responds under varying conditions and compare its behavior with that of the baseline systems. By structuring our analysis around these central aspects, we aim to provide a comprehensive understanding of the model’s strengths, limitations, and potential areas for further improvement.

- Overall results
- Results by content length
- Results by domain
- Results by question type

When the model was trained using various combinations that incorporated structural information, a clear performance pattern emerged. In particular, the configuration that utilized both structured paragraph representations and enriched question formulations demonstrated a marked advantage over the other setups. This model significantly outperformed the previously developed baseline systems across both the F1 score and the EM score, indicating not only higher overall accuracy but also a higher rate of perfectly correct predictions. The comparative performance of the different training strategies, including those relying solely on baseline data, those enhanced with structured paragraph

information, and those further augmented with enriched question data, is summarized in detail in Table 7.1. In addition, Figure 7.1 provides a visual illustration of these results, making it easier to observe how the inclusion of structural paragraph data and enriched question data contributes to the overall improvement in model performance.

Table 7.1: *Performance of BERT Baseline, ChatGPT Baselines, QModel (Question Enrichment), KBModel (Structured Content + Question Enrichment)*

Model	Detail	EM	F1
Baseline1	Bert baseline model	0.49	0.57
Baseline2	ChatGPT 3.5 baseline model	0.45	0.56
Baseline3	ChatGPT 4.1 baseline model	0.57	0.67
Baseline4	Qwen2 baseline model	0.34	0.44
QModel	Model with enrichment question	0.52	0.60
KBModel	Model with content structure data + enrichment question (Call as knowledgeBase model)	0.60	0.68

Two distinct training strategies were employed when developing the model using structured paragraph data and enriched question data. In the first configuration, QModel was trained exclusively on enriched question data, without incorporating any additional structural information from the source paragraphs. In the second configuration, called KBModel (Knowledge Base Model), the training data combined the enriched questions with structured paragraph representations.

Results show that the second approach, which integrates structured paragraph data with enriched questions, delivers the highest overall performance among all tested models. When evaluated against the BERT baseline, this combined model achieves an 11% absolute improvement in precision for both EM and F1 scores. Relative to the ChatGPT 3.5 baseline, the KBModel demonstrates a 15% increase in EM accuracy and a 12% improvement in F1, indicating that the enriched structure and question formulation together provide a clear advantage over purely pretrained language model capabilities. Even when

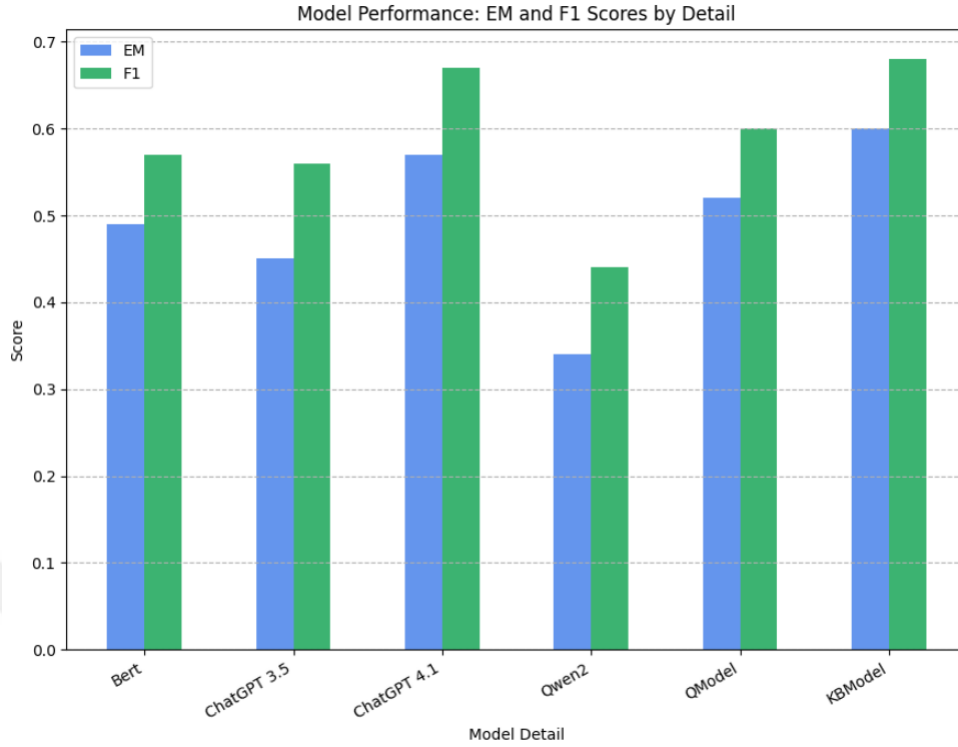


Figure 7.1: Performance of BERT baseline, ChatGPT baselines, Qwen2 baseline, QModel (Question Enrichment), KBModel (Structured Content + Question Enrichment)

compared to the very strong ChatGPT 4.1 baseline, KBModel still attains superior results, with a 3% improvement in EM and a 1% gain in F1. Furthermore, relative to the Qwen2 baseline, the model trained with structured data and enriched question records a 25% increase in both EM and F1 scores.

To summarize these findings in terms of baselines and proposed models, baseline 1 (BERT) achieves slightly higher EM and F1 scores than baseline 2 (ChatGPT 3.5), despite ChatGPT 3.5 being a more recent model. Baseline 4, based on Qwen2, yields the lowest overall scores and consistently underperforms the other baseline systems. So it is not treated as a strong or competitive reference point in the subsequent error analysis. Introducing the QModel, which relies solely on Enrichment Questions, achieves better performance than the corresponding baselines, demonstrating that even without structured paragraph information, enhanced question formulation can improve EM and F1 metrics.

However, baseline 3 (ChatGPT 4.1) still surpasses the QModel in both EM and F1 scores, confirming that an advanced large language model can outperform a system trained only on enriched questions when no structured paragraph data are included. The situation changes markedly when structured content is integrated. The KBModel, which combines Structured Data with Enrichment Questions, substantially outperforms every other model baseline and proposed variants alike. This consistent superiority across multiple metrics and baselines suggests that the joint use of structured representations of the source paragraphs together with carefully enriched question data provides the most informative training signal. In practical terms, these results indicate that models designed to exploit both structured content and enriched queries are likely to offer the best performance for tasks requiring precise and reliable EM and F1 outcomes.

7.1 Results by Content Length

Table 7.2 compares the KBModel with three baselines in terms of performance metrics, EM, and F1 scores. Data were categorized according to token count perspectives (content, question, answer) and thresholds such as <95 for content, <7 for question, =1, =2, and =3 for the answer.

Content (c), Token Range: < 95: KB Model achieves the best performance, improving **EM by 6%** and **F1 by 6%** compared to Baseline1. It also increased the EM score by 3% and F1 score by 1% compared with the most successful Baseline 3 (ChatGPT 4.1) model.

Content (c), Token Range: >= 95: KB Model achieves the best performance, improving **EM by 13%** and **F1 by 15%** compared to Baseline1. When we compare according to Baseline 3, while the EM match scores are the same, the F1 score is 2% lower.

Question (q), Token Range: < 7: KB Model achieves the best performance,

improving **EM by 9%** and **F1 by 8%** compared to Baseline1. It also increased the EM score by 3% and the F1 score by 2% compared with Baseline 3.

Question (c), Token Range: ≥ 7 : KB Model achieves the best performance, improving **EM by 9%** and **F1 by 9%** compared to Baseline1. When we compare according to Baseline 3, while the EM is better than +1%, the F1 score is 3% lower.

Answer (c), Token Range: = 1: KB Model achieves the best performance, improving **EM by 13%** and **F1 by 11%** compared to Baseline1. It also increased the EM score by 4% and the F1 score by 2%.

Answer (c), Token Range: = 2: KB Model achieves the best performance, improving **EM by 18%** and **F1 by 17%** compared to Baseline1. Compared with Baseline 3, the EM score showed a 3% lower performance, and the F1 score showed a 6% lower performance.

Answer (c), Token Range: = 3: KB Model achieves the best performance, improving **EM by 17%** and **F1 by 17%** compared to Baseline1. Compared with Baseline 3, the EM score showed a 1% lower performance, and the F1 score showed a 2% lower performance.

Table 7.2: KB model vs baselines with tokenize count perspective as c =content, q = question, a = answer

B1	EM	F1	B2	EM	F1	B3	EM	F1	B4	EM	F1	Model	EM	F1	#Token	Type
Bert	0.54	0.62	GPT 3.5	0.44	0.56	GPT 4.1	0.57	0.67	Qwen2	0.33	0.44	KB Model	0.60	0.68	< 95	c
Bert	0.45	0.51	GPT 3.5	0.45	0.56	GPT 4.1	0.58	0.68	Qwen2	0.34	0.44	KB Model	0.58	0.66	≥ 95	c
Bert	0.52	0.60	GPT 3.5	0.54	0.57	GPT 4.1	0.58	0.66	Qwen2	0.32	0.43	KB Model	0.61	0.68	< 7	q
Bert	0.49	0.56	GPT 3.5	0.44	0.56	GPT 4.1	0.57	0.68	Qwen2	0.34	0.44	KB Model	0.58	0.65	≥ 7	q
Bert	0.54	0.60	GPT 3.5	0.47	0.58	GPT 4.1	0.63	0.73	Qwen2	0.34	0.44	KB Model	0.67	0.71	= 1	a
Bert	0.43	0.52	GPT 3.5	0.49	0.60	GPT 4.1	0.64	0.75	Qwen2	0.33	0.43	KB Model	0.61	0.69	= 2	a
Bert	0.37	0.47	GPT 3.5	0.42	0.57	GPT 4.1	0.55	0.66	Qwen2	0.34	0.45	KB Model	0.54	0.64	= 3	a

KB Model outperforms both baselines significantly, achieving an **EM improvement of 6-18%** and an **F1 improvement of 6-17%**. The KB Model consistently outperformed both Baseline1 (BERT) and Baseline2 (ChatGPT) across all token ranges and

types, particularly in answer-focused tasks. The performance of all models varies across token ranges; however, the KB Model maintains its superiority in handling both short and long token ranges effectively. If we evaluate it in terms of token count according to Baseline3, KB Model has achieved more successful results in short texts and EM scores.

7.2 Results by Domain

The Table 7.3 compares the performance of five models: Baseline1 (BERT), Baseline2 (ChatGPT 3.5), Baseline3 (ChatGPT 4.1), Baseline4(Qwen2) and KB model in various domains using two evaluation metrics: EM and F1 score. The order in Table 7.3 is created according to the domains in where baseline1 is the most successful and the least successful. Against these domains, the KB model achieved more successful results in many domains than in all the baselines. In the domains where Baseline 1 and Baseline 3 were successful (Rank 1-5), the KB model achieved the most successful results. In addition, the KB model achieved more success in the domains where Baseline 1 and Baseline 3 were the least successful.

Baseline1 Top 5 Domain : KB Model achieves the best performance, improving **EM by 8%** and **F1 by 14%** compared to Baseline1. It also achieves the best performance, improving **EM by 18%** and **F1 by 17%** compared to Baseline3.

Baseline1 Down 5 Domain : KB Model achieves the best performance, improving **EM by 18%** and **F1 by 11%** compared to Baseline1. It also achieves the best performance, improving **EM by 14%** and **F1 by 10%** compared to Baseline3.

The performance of all models varies domain-by-domain, but The KB model offers more successful and balanced results. When we evaluated ChatGPT 3.5 and ChatGPT 4.1, we observed no consistent improvement as far as the number of tokens was concerned, and in some cases, ChatGPT 3.5 also produced more accurate results than ChatGPT 4.1.

7.3 Results by Question Type

Table 7.4 evaluates the performance of the five Baseline1 (BERT), Baseline2 (ChatGPT 3.5), Baseline3 (ChatGPT 4.1), Baseline4(Qwen2) and KBModel across various types of questions using the following metrics: EM and F1 score. Table 7.4 was created according to the seven types of questions (number of each type > 100), with ranking of Baseline1 being the most successful. The success of the KB model was evaluated against the 7 types of questions.

- **Nc Zaman (When):**

- BERT performs moderately EM (0.76) and F1 (0.80).
- KB Model achieves the highest EM (0.81) but is lower in F1 (0.75).
- ChatGPT 3.5 performs moderately (EM: 0.72, F1: 0.60).
- ChatGPT 4.1 achieves the highest F1 (EM: 0.75, F1: 0.85).

- **Kaç (How Much):**

- BERT follows with EM (0.69) and F1 (0.73).
- KB Model achieves the highest scores (EM: 0.73, F1: 0.75).
- ChatGPT 3.5 lags significantly (EM: 0.48, F1: 0.33).
- ChatGPT 4.1 lags significantly (EM: 0.47, F1: 0.63).

- **Ne Kadar (How Many):**

- BERT scores EM (0.62) and F1 (0.67).
- KB Model follows with EM (0.57) and F1 (0.65).
- ChatGPT 3.5 struggles with EM (0.49) and F1 (0.34).
- ChatGPT 4.1 struggles with EM (0.46) and F1 (0.63).

- **Kim (Who):**

- BERT scores EM (0.57) and F1 (0.63).
- KB Model follows ChatGPT 4.1 with small differences (EM: 0.72, F1: 0.77).
- ChatGPT 3.5 scores are lower than all models (EM: 0.55, F1: 0.43).
- ChatGPT 4.1 scores are the most successful (EM: 0.73, F1: 0.79).

- **Nerede (Where):**

- BERT scores EM (0.45) and F1 (0.52).
- KB Model achieves the highest EM (EM: 0.64, F1: 0.58).
- ChatGPT 3.5 performs poorly (EM: 0.43, F1: 0.35).
- ChatGPT 4.1 achieves the highest F1 (EM: 0.51, F1: 0.63).

- **Hangi (Which):**

- BERT scores EM (0.51) and F1 (0.59).
- KB Model performs as ChatGPT 4.1 for EM (0.65) and F1 (0.62).
- ChatGPT 3.5 lags with EM (0.54) and F1 (0.46).
- ChatGPT 4.1 achieves the highest F1 (0.65) and F1 (0.73).

- **Ne (What):**

- BERT achieves EM (0.42) and F1 (0.52).
- KB Model achieves the highest EM (0.61) and F1 (0.53).
- ChatGPT 3.5 has a similar EM (0.48) but lower F1 (0.36).
- ChatGPT 4.1 achieves the highest F1 (0.50) but lower F1 (0.61).

Overall Performance: KB Model consistently performs well across question types for EM, and surpasses Baseline1, Baseline2 and Baseline4. KB Model excels particularly in EM scores for "Kim (Who)" and "Nerede (Where)."

Table 7.3: KB model vs baselines - Domain based evaluation

Domain	B1	EM	F1	B2	EM	F1	B3	EM	F1	B4	EM	F1	Model	EM	F1	B1 Rank
Islamism	Bert	0.72	0.77	GPT 3.5	0.72	0.77	GPT 4.1	0.51	0.62	Qwen2	0.31	0.41	KB Model	0.69	0.76	1
1973 oil crisis	Bert	0.68	0.74	GPT 3.5	0.48	0.60	GPT 4.1	0.61	0.66	Qwen2	0.45	0.53	KB Model	0.68	0.75	2
Kenya	Bert	0.66	0.76	GPT 3.5	0.43	0.56	GPT 4.1	0.44	0.54	Qwen2	0.28	0.43	KB Model	0.60	0.71	3
Harvard University	Bert	0.63	0.63	GPT 3.5	0.61	0.71	GPT 4.1	0.66	0.73	Qwen2	0.51	0.57	KB Model	0.71	0.77	4
Sky (United Kingdom)	Bert	0.62	0.66	GPT 3.5	0.50	0.62	GPT 4.1	0.65	0.74	Qwen2	0.36	0.43	KB Model	0.70	0.77	5
Civil disobedience	Bert	0.38	0.47	GPT 3.5	0.44	0.50	GPT 4.1	0.46	0.57	Qwen2	0.34	0.40	KB Model	0.47	0.52	44
European Union law	Bert	0.36	0.53	GPT 3.5	0.36	0.52	GPT 4.1	0.52	0.62	Qwen2	0.26	0.42	KB Model	0.55	0.64	45
Prime number	Bert	0.35	0.43	GPT 3.5	0.28	0.40	GPT 4.1	0.47	0.59	Qwen2	0.23	0.38	KB Model	0.37	0.49	46
Chloroplast	Bert	0.34	0.48	GPT 3.5	0.26	0.38	GPT 4.1	0.38	0.51	Qwen2	0.18	0.32	KB Model	0.52	0.59	47
Computational theory	Bert	0.34	0.51	GPT 3.5	0.26	0.38	GPT 4.1	0.42	0.54	Qwen2	0.14	0.25	KB Model	0.46	0.56	48

Table 7.4: KB model vs baselines with question type perspective, q type count ≥ 100

Question Type	B1	EM	F1	B2	EM	F1	B3	EM	F1	B4	EM	F1	Model	EM	F1	B1 Rank
Ne zaman (When)	Bert	0.76	0.80	GPT 3.5	0.72	0.60	GPT 4.1	0.75	0.85	Qwen2	0.27	0.43	KB Model	0.81	0.75	1
Kaç (How much)	Bert	0.69	0.73	GPT 3.5	0.48	0.33	GPT 4.1	0.47	0.63	Qwen2	0.24	0.33	KB Model	0.73	0.65	2
Ne kadar (How much)	Bert	0.62	0.67	GPT 3.5	0.49	0.34	GPT 4.1	0.46	0.63	Qwen2	0.19	0.31	KB Model	0.57	0.44	3
Kim (Who)	Bert	0.57	0.63	GPT 3.5	0.72	0.62	GPT 4.1	0.73	0.79	Qwen2	0.57	0.65	KB Model	0.77	0.73	4
Nerede (Where)	Bert	0.45	0.52	GPT 3.5	0.55	0.43	GPT 4.1	0.51	0.63	Qwen2	0.19	0.34	KB Model	0.64	0.58	5
Hangi (Which)	Bert	0.51	0.59	GPT 3.5	0.65	0.54	GPT 4.1	0.65	0.73	Qwen2	0.46	0.56	KB Model	0.69	0.62	6
Ne (What)	Bert	0.42	0.52	GPT 3.5	0.48	0.36	GPT 4.1	0.50	0.61	Qwen2	0.28	0.39	KB Model	0.61	0.53	7

7.4 Error Analysis

An in-depth error analysis involves the examination of error results from various perspectives. This includes a thorough investigation of the question structure, expected responses, methodologies of BERT-based models, and behavior patterns of ChatGPT (generative AI), particularly through failure cases. The primary focus of this study and the

dataset was a QA system for reading comprehension. Given that this study focused on reading comprehension in Turkish, error analyses were interpreted within the context of this task. Within the test set, instances of model failure were meticulously examined individually, allowing the categorization of certain errors into specific groups. Observations of these errors have been documented. The BERT model utilizes span classification, whereas ChatGPT operates on a generative model framework, which can impact results in both positive and negative ways. Before conducting a comprehensive analysis, it is essential to provide a general evaluation of the two models. The Qwen2 model produced less accuracy than all other models so this model was not detailed in the error analysis study.

- In terms of performance, the BERT model demonstrates a significantly faster response time.
- Although the precision of the BERT model (KBModel) is higher than other baselines. GPT 3.5 results tend to be more unpredictable and varied than GPT 4.1.
- Error interpretation and detection are more straightforward within BERT base models, whereas this process is more complex with ChatGPT.
- In particular, BERT models do not exhibit the risk of hallucination, while instances of hallucinations have been identified in failure cases involving ChatGPT.

Errors can be classified into several categories, including those specific to the Turkish language, difficulties associated with grammatical structures, the impacts of punctuation and spelling, and variations arising from different modeling approaches. Numerous error analyses have encompassed microstudy areas that stand on their own.

Error Case 1: Generative models (GPTs) tend to return all types of entities that are most similar to the insight they get from the question.

- *Question:* Super Bowl 50 devre arası şovunu hangi grup manşete taşıdı? (Which band headlined the Super Bowl 50 halftime show?)
- *Correct Answer:* Coldplay
- *KB-Model Predict:* Coldplay
- *ChatGPT Predict:* Coldplay, Beyoncé ve Bruno Mars

Error Case 2: In machine translation, certain answers in the paragraph are displayed as numbers, while the response presents them in written form. This inconsistency leads to challenges in the assessment of the accuracy of a given answer.

- *Question:* Von Miller maç sırasında kaç top kaptı? (How many tackles did Von Miller get during the game?)
- *Correct Answer:* 5
- *KB-Model Predict:* beş (five)
- *ChatGPT Predict:* Beş (five)

Error Case 3: Models have difficulty with questions like "who were they" and "which were they". While Bert returns its first entity, OpenAI tends to return all the entities.

- *Question:* Super Bowl şampiyonluğunu savunan kimlerdi? (Who were the defending Super Bowl champions?)
- *Correct Answer:* New England Patriots
- *KB-Model Predict:* New England Patriots
- *ChatGPT Predict:* Denver Broncos ve New England Patriots

Error Case 4: Since Turkish is an agglutinative language, models have difficulty in using affix-based directions in questions. ("kimi" not equal "kim" in Turkish but "who" equals "kimi" and "kim" in English.)

- *Question 1:* Panthers NFC şampiyonluk maçında **kimi** yendi? (Who did the Panthers beat in the NFC championship game?)
- *Correct Answer:* Arizona Cardinals
- *KB-Model Predict:* New England Patriots
- *ChatGPT Predict:* Arizona Cardinals
- *Question 2:* AFC şampiyonası'nda Broncos'a **kim** yenildi? (Who lost to the Broncos in the AFC championship?)
- *Correct Answer:* New England Patriots
- *KB-Model Predict:* New England Patriots
- *ChatGPT Predict:* Arizona Cardinals

Error Case 5: In cases where ChatGPT cannot find the answer, it can give hallucinatory answers.

- *Question :*2015 NFL sezonunun en değerli oyuncusu kimdi? (Who was the most valuable player of the 2015 NFL season?)
- *Correct Answer:* Cam Newton
- *KB-Model Predict:* Cam Newton
- *ChatGPT Predict:* 2015, NFL, sezon, en değerli oyuncu, kim? (2015, NFL, season, Most Valuable Player, who?)

Error Case 6: Models have difficulty with questions involving inference, counting, or ordering.

- *Question:* Düşünülen üçüncü şehir neydi? (What was the third city considered?)
- *Correct Answer:* San Francisco
- *KB-Model Predict:* New Orleans
- *ChatGPT Predict:* New Orleans, Miami ve San Francisco (New Orleans, Miami and San Francisco)

Error Case 7: Models may encounter a dilemma when responding to inquiries that could be deemed accurate in either situation. This outcome adversely affects the EM when a conclusive answer is sought.

- *Question:* WBT-FM hangi şehirde bulunur? (What city is WBT-FM located in?)
- *Correct Answer:* Chester, Güney Carolina (Chester, South Carolina)
- *KB-Model Predict:* Chester
- *ChatGPT Predict:* Chester, Güney Carolina (Chester, South Carolina)

Error Case 8: Models struggle with inquiries regarding the first or last. Owing to bias, the nearest semantic entity to the statement may be provided as an answer.

- *Question:* California en son ne zaman Super Bowl'a ev sahipliği yaptı? (When was the last time California hosted the Super Bowl?)
- *Correct Answer:* 2003
- *KB-Model Predict:* 2003
- *ChatGPT Predict:* 2003 ve 2014

Error Case 9: The model might struggle to identify the block when the proper nouns are mixed with the common nouns.

- *Question:* NFL’de hangi takım şimdiye kadarki en iyi başlangıcı yaptı? (Which team has had the best start ever in the NFL?)
- *Correct Answer:* Carolina Panthers
- *KB-Model Predict:* Carolina
- *ChatGPT Predict:* Panthers

Error Case 10: In generative models, formatting mistakes can happen because of insufficient reliance on the text.

- *Question:* Newton 2015’te kaç yard attı? (How many yards did Newton throw for in 2015?)
- *Correct Answer:* 3.837
- *KB-Model Predict:* 3.837
- *ChatGPT Predict:* 3837

Error Case 11: The generative model offered a superior fit for the outcomes related to the synonymy between the context and the response following the Turkish translation.

- *Question:* Broncos’un koçu kim? (Who is the head coach of the Broncos?), Head coach = Koç, Antrenör in Turkish but question contains ”Koç”, content contains ”Antrenör”.
- *Correct Answer:* Gary Kubiak

- *KB-Model Predict:* Wade Phillips
- *ChatGPT Predict:* John Fox ve Gary Kubiak

Error Case 12: Since Turkish is an agglutinative language, the responses generated by the models include suffixes. This influences the EM score.

- *Question:* Thomas Davis NFC Şampiyonluk Oyunu sırasında hangi vücut parçasını kırdı? (What body part did Thomas Davis break during the NFC Championship Game?)
- *Correct Answer:* kol (arm)
- *KB-Model Predict:* kolla (with arm)
- *ChatGPT Predict:* Kırık kolu (broken arm)

Error Case 13: It has been observed that a broad and comprehensive LLM model (ChatGPT) recognizes and interprets some technical terms better.

- *Question :*50. Super Bowl'un isimlendirilmesinde Roma rakamları kullanılsaydı hangisi kullanılırdı? (If Roman numerals were used in naming Super Bowl 50, which would they be?)
- *Correct Answer:* L
- *KB-Model Predict:* Super Bowl 50
- *ChatGPT Predict:* L

Error Case 14: In the answers sought in or outside the parentheses, the models focus more on returning all the data, in which case it affects the EM.

- *Question:* Program kaç kg ay taşını geri getirdi? (How many kg of moon rocks did the program bring back?)
- *Correct Answer:* 382 kg
- *KB-Model Predict:* 842 pound (382 kg)
- *ChatGPT Predict:* 842 pound (382 kg)

Error Case 15: When asked when it ended, it started and how long it lasted in historical data given over a period of time, models generally provide the correct answer indirectly.

- *Question:* Watt, Newcomen'in motorundaki geliřtirmelerinin geliřimini ne zaman bitirdi? (When did Watt complete her improvements on Newcomen's engine?)
- *Correct Answer:* 1775
- *KB-Model Predict:* 1763-1775
- *ChatGPT Predict:* 1763-1775

Error Case 16: It is one of the factors that affect the full accuracy in open-ended situations in the dataset.

- *Question:* Varřova'nın Eski Kent'i 1980'de hangi listeye yazıldı? (What list was Warsaw's Old Town included in 1980?)
- *Answer:* UNESCO'nun Dünya Mirası listesi (UNESCO's World Heritage List)
- *KB-Model Predict:* UNESCO
- *ChatGPT Predict:* UNESCO

- *Question:* Kimin tezi, bir sorunun çözümünün polinom zaman algoritmasına izin verdiğini varsayarak makul kaynaklarla çözülebilir olduğunu belirtir? (Whose thesis states that a problem is solvable with reasonable resources assuming that the solution allows a polynomial time algorithm?)
- *Answer:* Cobham'ın tezi (thesis of Cobham)
- *KB-Model Predict:* Cobham
- *ChatGPT Predict:* Cobham

Error Case 17: Models are generally difficult to answer, especially in formula-based questions that require deep scientific knowledge and interpretation skills.

- *Question :*Polinom süresi NP ile tamamlanmış bir sorun içinde kullanılabiliyorsa, P'nin neye eşit olduğu anlamına gelmez? (If polynomial time can be used in an NP-complete problem, does it not imply that P is equal to what?)
- *Answer:* NP
- *KB-Model Predict:* X sorunu C (X-C problem)
- *ChatGPT Predict:* $P = NP$

7.5 Performance and Cost Analysis

All models were evaluated on the SQuAD test set, which contains roughly 8,000 questions. For all experiments, we employed a zero-shot prompting strategy. In zero-shot prompting, the language model is asked to carry out a task without being given explicit, task-specific example inputs and outputs. This approach has several benefits: it is quick to configure, requires no collection or curation of labeled examples, and is therefore particularly suitable for novel tasks or situations where only ad-hoc analyses are

needed. A detailed comparison of model performance and associated computation costs obtained under these conditions is presented in Table 7.5.

From a performance perspective, the BERT-based architectures (Baseline1 and KBModel) exhibit very low latency and thus high response-time efficiency. In addition, these models are economically advantageous because they can be deployed fully self-hosted on standard end-user hardware, avoiding recurring cloud inference charges. By contrast, ChatGPT-3.5 exhibits higher latency and higher usage costs relative to the BERT-based models. When compared with ChatGPT-4.5, ChatGPT-3.5 is both faster and less expensive, making it a more practical option when advanced capabilities are not strictly required. Qwen2-7B, as an open-source model, offers an additional alternative: when self-hosted, it provides substantially reduced operating costs and faster inference.

Table 7.5: *Performance of models 8K question call for test dataset*

Model	Total Response Time	Cost (API or self-host)
GPT-3.5	28 Hours	25 (Open AI)\$
GPT-4.1	63 Hours	200 (Open AI) \$
Qwen2-7B-Instruct	14 Hours	1 \$ (Google colab)
BERT	3 Hours	self-host
KBModel	3 Hours	self-host + (Graph Creation: 100\$)

7.6 Alternative Dataset Performance

As an additional benchmark to KBModel on the mSQuAD dataset, we also evaluated its performance on the existing TQuADv1 dataset. This dataset served to assess how KBModel performs relative to other models when trained and tested on TQuADv1 [6]. TQuADv1 was originally constructed as a Turkish reading-comprehension resource. Nevertheless, its domain is restricted to Turkish and Islamic Science History, and it includes a relatively small number of questions. Specifically, the test split comprises only 892 questions. We applied KBModel to this dataset and then compared its outputs with

the results reported in [43], which had previously experimented with the same dataset. In this comparative evaluation, KBModel produced superior performance.

In TQuADv1, the model achieved an EM score of 0.60 and an F1 score of 0.77, indicating strong accuracy despite the dataset’s limited size and narrow topical scope. Because the dataset covers a constrained set of subjects and includes relatively few questions, we did not perform a detailed analysis, such as domain-level, etc. Instead, we focused on aggregate metrics. In the study by [43], various fine-tuning strategies were applied to several baseline models, namely, BERTurk-base, T5-small, T5-base, and T5-large, on the TQuADv1 dataset. Our study directly compares the evaluation results of KBModel with those of baseline models in Table 7.6.

Table 7.6: *Performance of KBModel on TQuADv1 dataset and comparison of its base-lines*

Model	EM	F1
BERTurk-base	0.45	0.62
MT-Both T5-small	0.48	0.63
MT-Both T5-base	0.55	0.72
MT-Both T5-large	0.59	0.74
KBModel	0.60	0.77

8. COLLABORATIVE WORK WITH LINGUISTIC RESEARCHERS

Our mSQuAD dataset studies served as input for various groups working in NLP. In this context, it was used as the primary dataset for the question-based dependency treebank. All shared questions were marked with dependency and consistency.

The treebank name is HaydiSor. It is based on the widely used annotation scheme of the Universal Dependencies (UD) project [44], which defines a unified format for syntactic and morphological annotation across languages. The UD framework currently covers roughly 100 languages and offers nine English treebanks, with some studies additionally providing parallel resources for other language pairs. Although these English treebanks have made substantial contributions to both linguistics and NLP, they include only a small fraction of interrogative sentences. HaydiSor is, to our knowledge, the first English UD treebank centered on a large collection of questions, thereby filling a significant gap in existing resources.

8.0.1 *Building the Corpus of the Treebank*

It is derived from the SQuAD 1.1 dataset, which includes about 72,000 sentences. For the HaydiSor treebank, we extracted a subset of 10,000 sentences. Because the full dataset is large, we needed to balance the intensive effort of manual annotation with sufficient coverage. We therefore selected a subset that captures the dataset’s diversity and structural characteristics. As illustrated in Figure 8.1, the distribution of question lengths in the full SQuAD dataset and in the HaydiSor treebank aligns closely, indicating that our sample provides a representative cross-section of the original data.

Before determining the final number of HaydiSor questions, we first reviewed the

¹The table’s findings display the initial RegEx-tagged queries. The questions were not all identified as a result. We reviewed those entries and made the necessary corrections later.

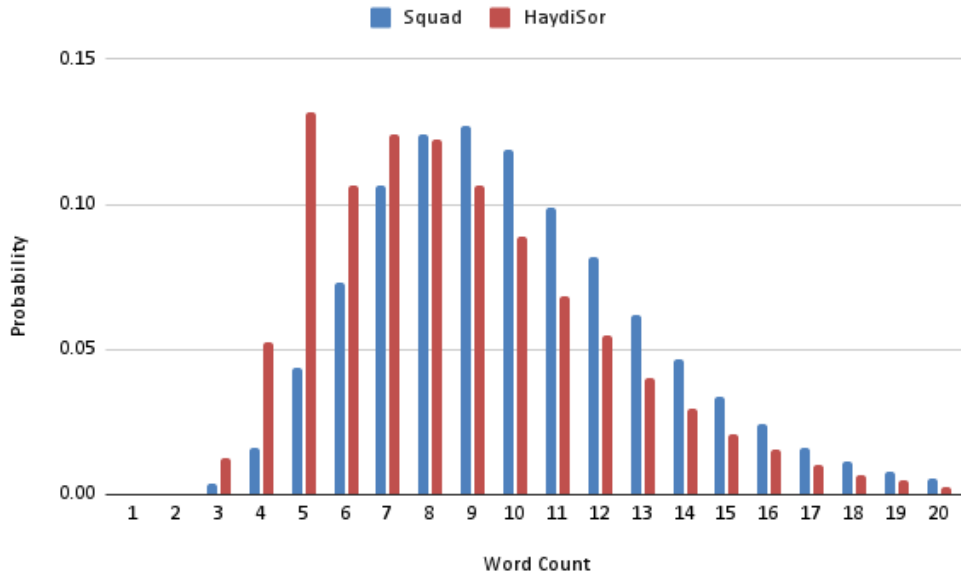


Figure 8.1: *The distribution of word counts of questions in Squad and HaydiSor*

entire set and corrected spelling errors, revising 3,100 sentences (4%) in total. We then examined the interrogative patterns in the corpus and extracted 188 distinct question types, with their initial distribution presented in Table 8.1. After sorting questions by length, we manually assigned each question to one of these types, obtaining 5,553 interrogative sentences.

We further annotated questions automatically with question types using both constituency and dependency parsers, which together produced 30,746 annotations. The constituency parser aligned more closely with the manually derived question types than the dependency parser (26,948 vs. 24,553 questions). Consequently, when reducing the number of sentences in the treebank, we prioritized annotations from the constituency parser over those from the dependency parser.

To select the 10,000 questions included in the treebank, we applied the following procedure. We started with three sources: 2,615 manually annotated questions, 9,600 questions annotated by the constituency parser, and 7,885 questions annotated by the

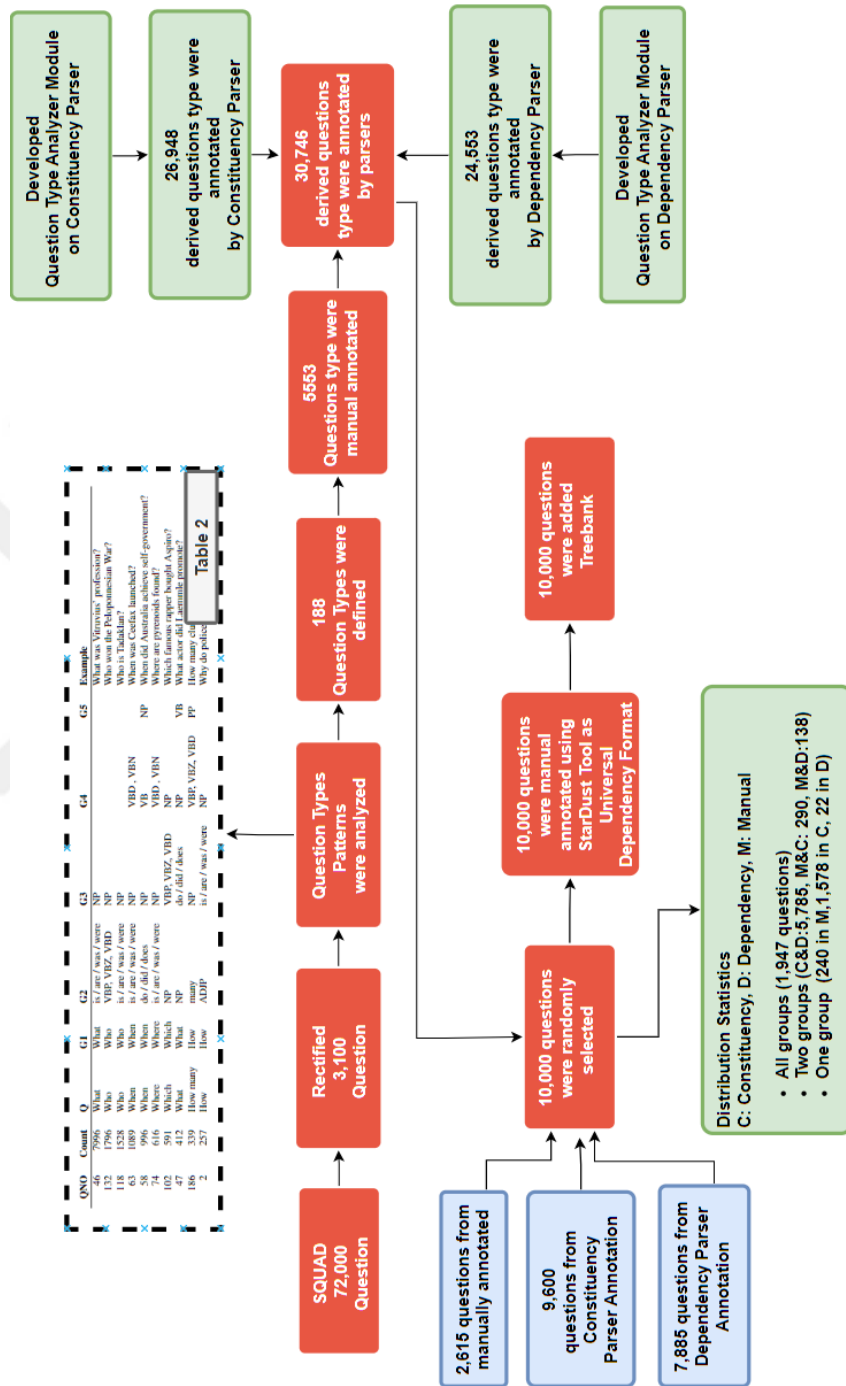


Figure 8.2: All processes are shown end to end

Table 8.1: *Distribution of questions in the corpus.*

Question	n_train	% of n_train
What	36,279	55.51%
Who	7,511	11.49%
Which	4,940	7.56%
When	4,803	7.35%
How many	3,778	5.78%
Where	2,851	4.36%
How	2,627	4.02%
How much	643	0.98%
Yes/No	506	0.77%
Why	500	0.77%
Whom	284	0.43%
Whose	269	0.41%
NA	247	0.38%
Imperative[1]	120	0.18%

dependency parser. Because of time constraints, the contribution of manual annotations is relatively small compared to the automatic resources. We then checked for overlapping entries between the constituency and dependency parsers. When no manual annotation existed for a sentence, we preferentially retained the constituency parser’s analysis, as it yielded slightly better agreement with the target question types. Nonetheless, questions for which only the dependency parser produced a match were also incorporated. Overall, 1,947 questions appear in all three groups; 5,785 in both constituency and dependency, 290 in manual and constituency, and 138 in manual and dependency; while 240 occur only in the manual set, 1,578 only in the constituency set, and 22 only in the dependency set. The full workflow is presented step by step in Figure 8.2.

Regarding the formatting of question types, we employed the Penn Treebank Part-of-Speech (PoS) tag set. Our linguists identified 188 distinct question types in the corpus, with group lengths of up to seven elements. These groups include phrases such as NP, PP, and AdjP; additionally, we further segmented phrases containing wh-elements, as

these form the core of the analysis. Table 8.2 presents the ten most frequently occurring question types.

8.0.2 Annotation

The final HaydiSor corpus comprises 10,000 sentences annotated for dependency structure according to the UD framework, and is thus provided in Computational Natural Language Learning (CoNLL) format. Our team used the StarDust interface [45] for annotation. In this interface, annotators draw a line from the dependent to the head to indicate dependency relations, then select an appropriate label from a pop-up menu listing the relevant tags, as illustrated in Figure 8.3. The resulting arrow originates from the root and points toward the dependent, with the dependency label displayed along the arrow. Hovering over a tag reveals previous annotations of that specific word with the same label.

StarDust can also validate root forms by checking them against Princeton WordNet 3.1 [46]. If a root is not found in WordNet, the editor highlights it in red to warn the annotator that it may be a typographical error or a proper noun absent from the lexicon. In addition, we used StarDust’s automatic annotation functionality for shallow parsing: frequent roles, such as subjects, verbs, and other common tags, were automatically assigned, while the remaining dependencies were annotated manually.

Table 8.2: Phrase groups and examples for the most 10 frequent question types

QNO	Count	Q	G1	G2	G3	G4	G5	Example
46	7996	What	What	is / are / was / were	NP			What was Vitruvius' profession?
132	1796	Who	Who	VBZ, VBZ, VBD	NP			Who won the Peloponnesian War?
118	1528	Who	Who	is / are / was / were	NP			Who is Tadaklan?
63	1089	When	When	is / are / was / were	NP	VBD, VBN		When was Ceefax launched?
58	996	When	When	do / did / does	NP	VB	NP	When did Australia achieve self-government?
74	616	Where	Where	is / are / was / were	NP	VBD, VBN		Where are pyrenoids found?
102	591	Which	Which	NP	VBZ, VBZ, VBD	NP		Which famous rapper bought Aspiro?
47	412	What	What	NP	do / did / does	NP	VB	What actor did Laemmle promote?
186	339	How many	How	many	NP	VBZ, VBZ, VBD	PP	How many clubs entered in 2005-06?
2	257	How	How	ADJP	is / are / was / were	NP		Why do police use radios?

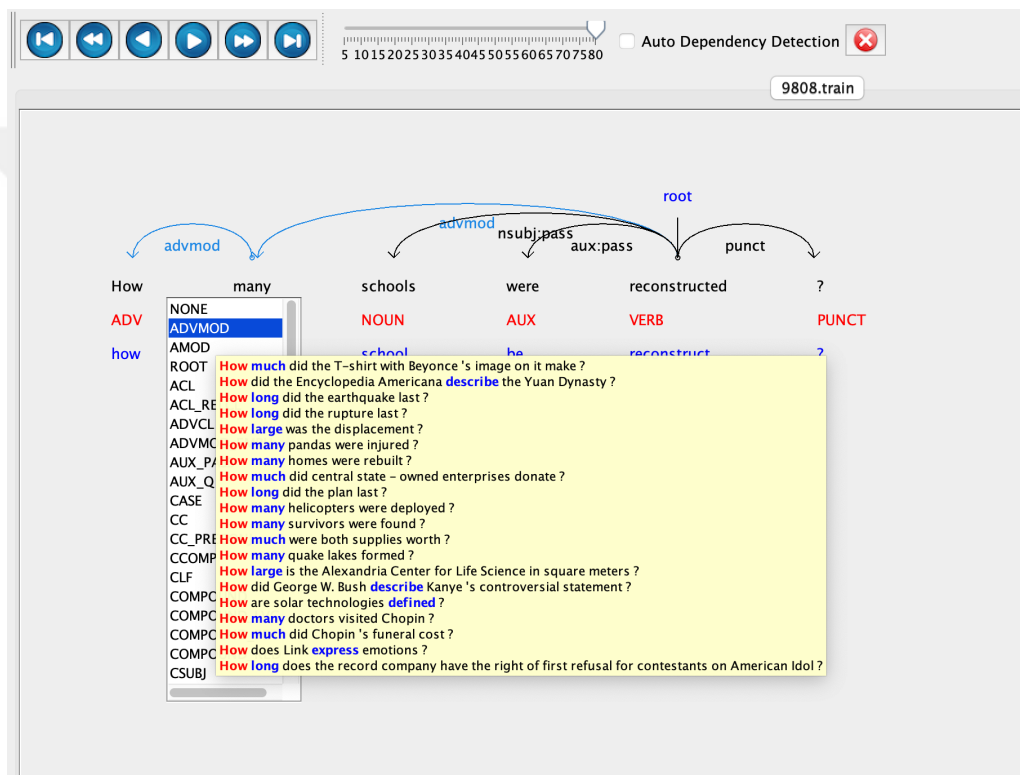


Figure 8.3: *The interface used to mark the dependency relations*

9. CONCLUSIONS

The KB Model substantially surpasses both Baseline1 (BERT) and Baseline2 (ChatGPT 3.5) across a wide spectrum of evaluation conditions, including different token length ranges, diverse question types, and multiple application domains. Specifically, it delivers performance gains of approximately 6–18% in EM and 6–17% in F1 scores. These improvements are particularly pronounced in answer-oriented tasks, where precise extraction and accurate matching are crucial. Moreover, the KB Model achieves marked advances in performance in both the top five and bottom five domains when compared with Baseline1, demonstrating that the model is not only strong in domains where baseline models already perform well, but also able to improve results in more challenging or underperforming areas.

When compared with Baseline3 (ChatGPT 4.1), the KB Model also shows generally superior results: its F1 score is higher by about 3%, and its EM score is increased by roughly 1%. These improvements emphasize the model’s effectiveness in tasks that require both accuracy and consistency. Nonetheless, there are limited cases in which Baseline3 achieves slightly better F1 scores, indicating that while the KB Model is overall stronger, certain domain-question combinations or specific task characteristics may still favor Baseline3. Baseline4 (Qwen2) produces less accurate results than all the other baselines and the KB Model, underscoring the KB-based approach’s competitiveness.

A critical contribution to the KB Model’s success comes from transforming the raw data into a more structured, machine-readable format and enriching the question representations with additional contextual information. This process of structuring and enrichment leads to notable performance gains in the baseline models. Results vary significantly across domains, particularly for long and complex input sequences (longer token lengths). As a consequence, the KB Model appears more balanced across domains and question categories, reducing performance volatility. The KB Model architecture captures

subtler relationships, edge cases, and domain-specific nuances.

The knowledge base is built on an infrastructure that combines LangChain, OpenAI tools, and ChatGPT, serving both as the underlying framework and the service layer. This infrastructure plays a central role in knowledge mapping, linking related concepts. Its design enables scalability and flexibility, and it can be extended to support multiple languages. When the underlying components are equipped with multilingual capabilities, the same framework can be deployed in other languages with relatively minimal adaptation. However, the infrastructure primarily operates as a Software-as-a-Service (SaaS) solution. This limitation makes open-source availability a significant focus for future work.

In error analyses, there are special cases where both span classification and generative AI are inadequate. There are areas of development for all the models that require more intelligence, such as inference, ordering, selection, and counting. In addition, varying accuracies have been observed across language structure, spelling, translation output format, and synonymy. However, each issue requires detailed research.

A reference study on the translation method was conducted using a few sample datasets in various languages. The problems and solutions arising from the translation method were shared. Simultaneously, the accuracy of the deep learning models was observed to improve with the structural format and the enrichment of the data obtained. It has been shown that effective results can be obtained through data structuring for LLMs and Deep Learning models in a black-box setting. It has shown the importance of structuring data in question-and-answer systems.

REFERENCES

- [1] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “Squad: 100,000+ questions for machine comprehension of text,” *arXiv preprint arXiv:1606.05250*, 2016.
- [2] K. Tutar and O. T. Yıldız, “Turkish question-answer dataset evaluated with deep learning,” in *2024 9th International Conference on Computer Science and Engineering (UBMK)*, pp. 101–105, IEEE, 2024.
- [3] K. Tutar and O. Taner Yıldız, “Developing a robust question and answer system for the turkish language utilizing deep learning techniques,” *IEEE Access*, vol. 13, pp. 197329–197344, 2025.
- [4] C. B. Gemirter and D. Goularas, “A turkish question answering system based on deep learning neural networks,” *Journal of Intelligent Systems: Theory and Applications*, vol. 4, no. 2, pp. 65–75, 2021.
- [5] E. Budur, R. Özçelik, D. Soylu, O. Khattab, T. Güngör, and C. Potts, “Building efficient and effective openqa systems for low-resource languages,” *Knowledge-Based Systems*, vol. 302, p. 112243, 2024.
- [6] F. Soygazi, O. Çiftçi, U. Kök, and S. Cengiz, “Thquad: Turkish historic question answering dataset for reading comprehension,” in *2021 6th international conference on computer science and engineering (UBMK)*, pp. 215–220, IEEE, 2021.
- [7] C. P. Carrino, M. R. Costa-Jussà, and J. A. Fonollosa, “Automatic spanish translation of squad dataset for multi-lingual question answering,” in *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pp. 5515–5523, 2020.
- [8] D. Charlet, G. Damnati, F. Bechet, G. Marzinotto, and J. Heinecke, “Cross-lingual and cross-domain evaluation of machine reading comprehension with squad and calor-quest corpora,” in *Proceedings of the 12th Conference on Language Resources and Evaluation (LREC 2020)*, pp. 5491–5497, 2020.
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, *et al.*, “Attention is all you need [j],” *Advances in neural information processing systems*, vol. 30, no. 1, pp. 261–272, 2017.
- [10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186, 2019.
- [11] M. E. Peters, “Deep contextualized word representations,” *arXiv preprint arXiv:1802.05365*, 2018.

- [12] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, *et al.*, “Improving language understanding by generative pre-training,” 2018.
- [13] M. S. I. Sakib, “What is chatgpt,” *ResearchGate*, 2023.
- [14] C. Derici, K. Çelik, A. Özgür, T. Güngör, E. Kutbay, Y. Aydın, and G. Kartal, “Rule-based focus extraction in turkish question answering systems,” in *2014 22nd Signal Processing and Communications Applications Conference (SIU)*, pp. 1604–1607, IEEE, 2014.
- [15] M. Hrou, “Evaluating squad-based question answering for the open research knowledge graph completion,” 2022.
- [16] O. Ciftci, F. Soygazi, and S. Tekir, “Enrichment of turkish question answering systems using knowledge graphs,” *Turkish Journal of Electrical Engineering and Computer Sciences*, vol. 32, no. 4, pp. 516–533, 2024.
- [17] A. Talmor and J. Berant, “The web as a knowledge-base for answering complex questions,” *arXiv preprint arXiv:1803.06643*, 2018.
- [18] R. Das, M. Zaheer, S. Reddy, and A. McCallum, “Question answering on knowledge bases and text using universal schema and memory networks,” *arXiv preprint arXiv:1704.08384*, 2017.
- [19] I. Alagha, “Leveraging knowledge-based features with multilevel attention mechanisms for short arabic text classification,” *IEEE Access*, vol. 10, pp. 51908–51921, 2022.
- [20] Q. Wu, P. Wang, C. Shen, A. Dick, and A. Van Den Hengel, “Ask me anything: Free-form visual question answering based on knowledge from external sources,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 4622–4630, 2016.
- [21] F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller, “Language models as knowledge bases?,” in *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pp. 2463–2473, 2019.
- [22] D. Savenkov and E. Agichtein, “When a knowledge base is not enough: Question answering over knowledge bases with external text data,” in *Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval*, pp. 235–244, 2016.
- [23] K. Barker, “Combining structured and unstructured knowledge sources for question answering in watson,” in *International Conference on Data Integration in the Life Sciences*, pp. 53–55, Springer, 2012.

- [24] L. Dong, F. Wei, M. Zhou, and K. Xu, “Question answering over freebase with multi-column convolutional neural networks,” in *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 260–269, 2015.
- [25] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, “Think you have solved question answering? try arc, the ai2 reasoning challenge,” *arXiv preprint arXiv:1803.05457*, 2018.
- [26] G. Lai, Q. Xie, H. Liu, Y. Yang, and E. Hovy, “Race: Large-scale reading comprehension dataset from examinations,” *arXiv preprint arXiv:1704.04683*, 2017.
- [27] M. Richardson, C. J. Burges, and E. Renshaw, “Mctest: A challenge dataset for the open-domain machine comprehension of text,” in *Proceedings of the 2013 conference on empirical methods in natural language processing*, pp. 193–203, 2013.
- [28] J. Weston, A. Bordes, S. Chopra, A. M. Rush, B. Van Merriënboer, A. Joulin, and T. Mikolov, “Towards ai-complete question answering: A set of prerequisite toy tasks,” *arXiv preprint arXiv:1502.05698*, 2015.
- [29] C. Zeng, S. Li, Q. Li, J. Hu, and J. Hu, “A survey on machine reading comprehension—tasks, evaluation metrics and benchmark datasets,” *Applied Sciences*, vol. 10, no. 21, p. 7640, 2020.
- [30] N. Varghese and M. Punithavalli, “A comprehensive survey on machine reading comprehension: Models, benchmarked datasets, evaluation metrics, and trends,” in *Congress on Intelligent Systems: Proceedings of CIS 2021, Volume 2*, pp. 1–15, Springer, 2022.
- [31] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [32] J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990.
- [33] Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994.
- [34] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [35] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [36] F. A. Gers, J. Schmidhuber, and F. Cummins, “Learning to forget: Continual prediction with lstm,” *Neural Computation*, vol. 12, no. 10, pp. 2451–2471, 2000.

- [37] K. Cho, B. v. Merrienboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder-decoder for statistical machine translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1724–1734, 2014.
- [38] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, “Empirical evaluation of gated recurrent neural networks on sequence modeling,” *arXiv preprint arXiv:1412.3555*, 2014.
- [39] Y. Liu and M. Zhang, “Neural network methods for natural language processing,” 2018.
- [40] K. Tutar, “Translated squad turkish dataset github url: <https://github.com/kadirtutar/datasettranslated>,” 2025.
- [41] S. Schweter, “Berturk-bert models for turkish,” *Zenodo*, 2020.
- [42] W. Yang, L. Some, M. Bain, and B. Kang, “A comprehensive survey on integrating large language models with knowledge-based methods,” *Knowledge-Based Systems*, p. 113503, 2025.
- [43] F. Ç. Akyön, A. D. E. ÇAVUŞOĞLU, C. Cengiz, S. O. Altinuc, and A. Temizel, “Automated question generation and question answering from turkish texts,” *Turkish Journal of Electrical Engineering and Computer Sciences*, vol. 30, no. 5, pp. 1931–1940, 2022.
- [44] J. Nivre, D. Zeman, F. Ginter, and F. Tyers, “Universal dependencies,” in *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Tutorial Abstracts*, 2017.
- [45] A. Yenice, N. Cesur, A. Kuzgun, and O. I. S. A. U.-b. D. A. T. Yıldız, “Proceedings of the 16th linguistic annotation workshop (LAW-xvi) within LREC2022,” pp. 79–84, pp. 79–84, 2022.
- [46] C. Fellbaum, “ed,” *WordNet: an electronic lexical database. MIT Press, Cambridge MA*, vol. 1, p. 998, 1998.

VITA

Kadir TUTAR

Academic Degrees:

- 2010 – Bachelor’s Degree, Computer Engineering, Dokuz Eylul University
- 2014 – Master of Science, Computer Engineering, Ege University
- 2026 – Doctor of Philosophy, Computer Engineering, Özyeğin University

Professional Experience:

- 2010–2011: Software Engineer, TEKSAV Technology
- 2011–2014: Software Engineer, ASELSAN & Phillip Morris via CMA Consultancy
- 2015–2016: Software Engineer, Turkcell Global Bilgi
- 2016–2024: Software Development Lead, Turkcell Technology
- 2024–Present: Software Development Manager, BOYNER

Publications:

- Tutar, Kadir, and Olcay Taner Yildiz. "Developing a Robust Question and Answer System for the Turkish Language Utilizing Deep Learning Techniques." IEEE ACCESS 13 (2025): 197329-197344.

- Tutar, Kadir, and Olcay Taner Yıldız. "Turkish Question-Answer Dataset Evaluated with Deep Learning." 2024 9th International Conference on Computer Science and Engineering (UBMK). IEEE, 2024.
- Tutar, Kadir, Murat Osman ÜNALIR, and Levent Toker. "SOSYAL AĞLAR ÜZERİNDE ONTOLOJİ TABANLI SEZGİ ANALİZİ İÇİN BİR UYGULAMA ÇERÇEVESİNİN GELİŞTİRİLMESİ." Pamukkale University Journal of Engineering Sciences 21.5 (2015).

